

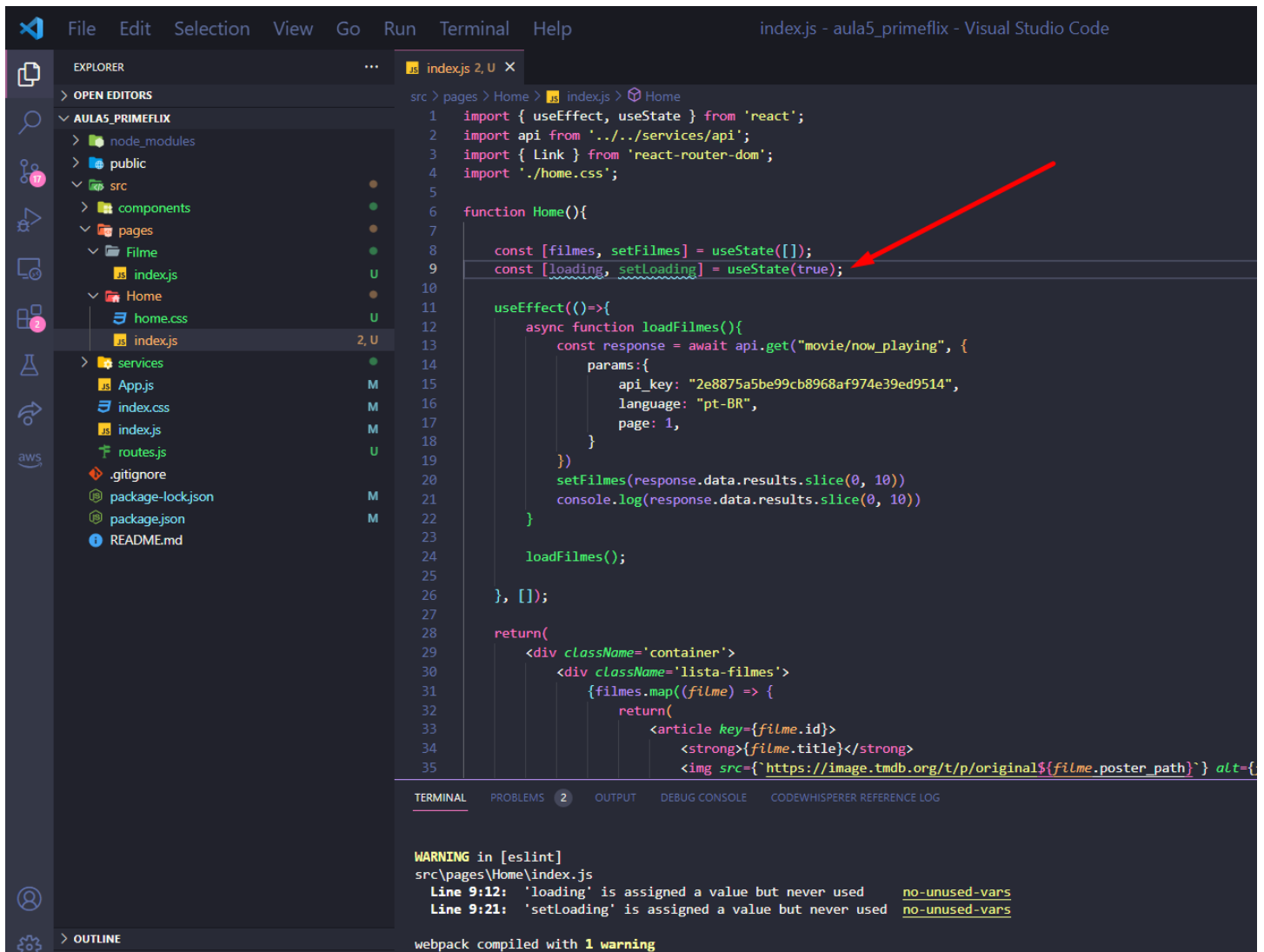
Aula 6 – Continuação Prime Flix

Objetivo da aula:

- Continuação da Aula 5
- Criando loading e Not Found
- Criando a página de cada filme automaticamente
- Utilizando o hook useNavigate
- Exibindo o trailer no youtube
- Salvando meus filmes favoritos
- Construindo a página de favoritos
- Estilizando Alerts

Criando um Loading

1 – Estando no arquivo index.js dentro da pasta Home, vamos implementar um novo useState do tipo boolean.



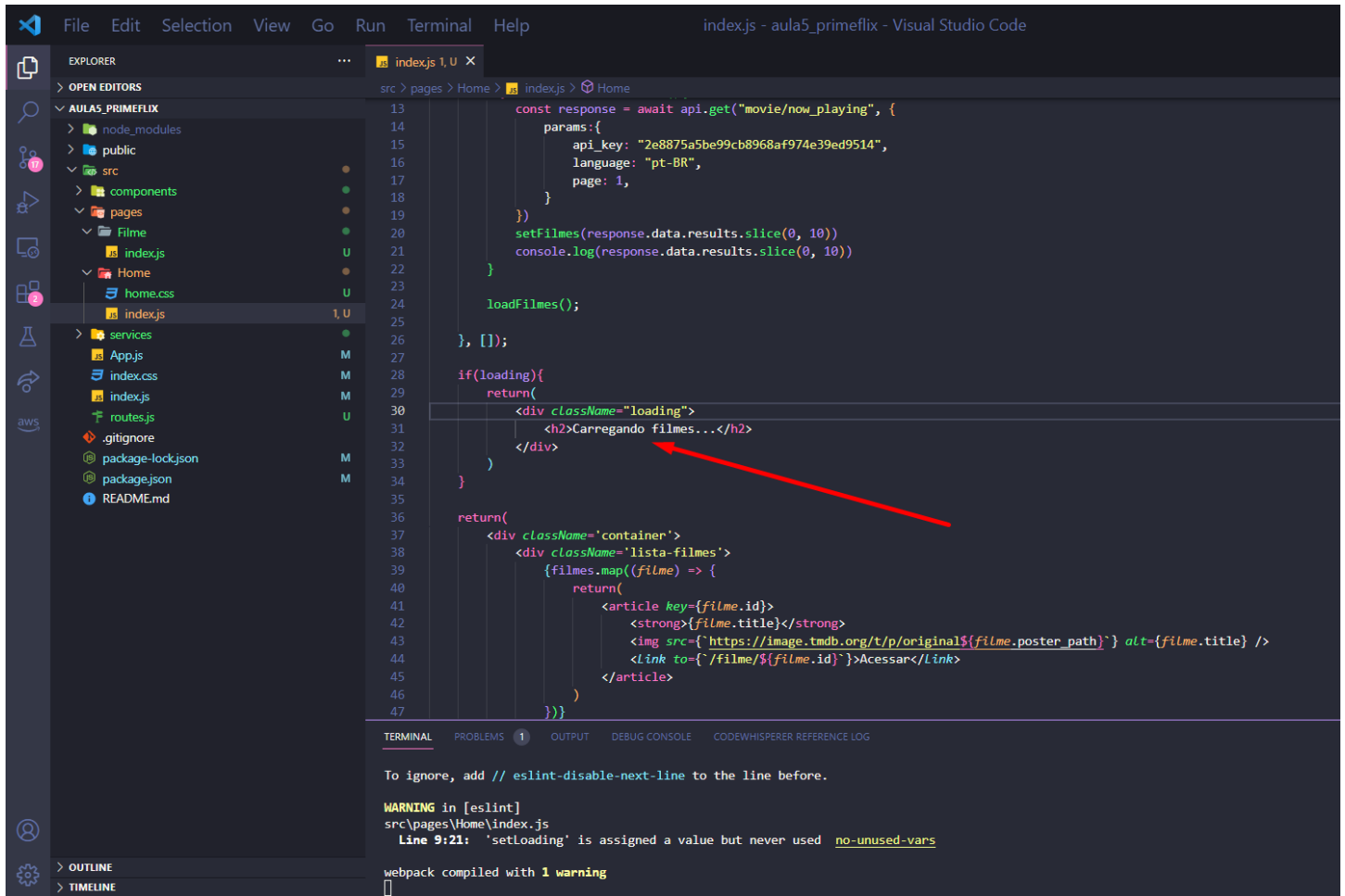
The screenshot shows the Visual Studio Code interface with the file explorer on the left, the editor in the center, and the terminal at the bottom. The file explorer shows the project structure with the 'index.js' file in the 'Home' directory selected. The editor displays the code for 'index.js', where a new state variable 'loading' is being added to the 'useState' hook. A red arrow points to the 'loading' variable in the state declaration. The terminal shows a warning from ESLint about unused variables.

```
1 import { useEffect, useState } from 'react';
2 import api from '../services/api';
3 import { Link } from 'react-router-dom';
4 import './home.css';
5
6 function Home() {
7
8   const [filmes, setFilmes] = useState([]);
9   const [loading, setloading] = useState(true);
10
11   useEffect(() => {
12     async function loadFilmes() {
13       const response = await api.get("movie/now_playing", {
14         params: {
15           api_key: "2e8875a5be99cb8968af974e39ed9514",
16           language: "pt-BR",
17           page: 1,
18         }
19       });
20       setFilmes(response.data.results.slice(0, 10));
21       console.log(response.data.results.slice(0, 10));
22     }
23
24     loadFilmes();
25
26   }, []);
27
28   return (
29     <div className='container'>
30       <div className='lista-filmes'>
31         {filmes.map((filme) => {
32           return (
33             <article key={filme.id}>
34               <strong>{filme.title}</strong>
35               <img src={`https://image.tmdb.org/t/p/original${filme.poster_path}`} alt={
```

WARNING in [eslint]
src\pages\Home\index.js
Line 9:12: 'loading' is assigned a value but never used no-unused-vars
Line 9:21: 'setloading' is assigned a value but never used no-unused-vars

webpack compiled with 1 warning

2 – Podemos entender que este estado será ou a página está em loading ou não está, portanto, adicione um if para verificar se o UseState é true, se for, adicionamos uma div com a frase “Carregando filme...”



```
13 const response = await api.get("movie/now_playing", {
14   params: {
15     api_key: "2e8875a5be99cb8968af974e39ed9514",
16     language: "pt-BR",
17     page: 1,
18   }
19 })
20 setFilmes(response.data.results.slice(0, 10))
21 console.log(response.data.results.slice(0, 10))
22 }
23 loadFilmes();
24 }, []);
25
26 if (loading) {
27   return (
28     <div className="loading">
29       <h2>Carregando filmes...</h2>
30     </div>
31   )
32 }
33
34 return (
35   <div className="container">
36     <div className="lista-filmes">
37       {filmes.map((filme) => {
38         return (
39           <article key={filme.id}>
40             <strong>{filme.title}</strong>
41             <img src={`https://image.tmdb.org/t/p/original${filme.poster_path}`} alt={filme.title} />
42             <Link to={`/filme/${filme.id}`}>Acessar</Link>
43           </article>
44         )
45       })}
46     </div>
47   )
48 }
```

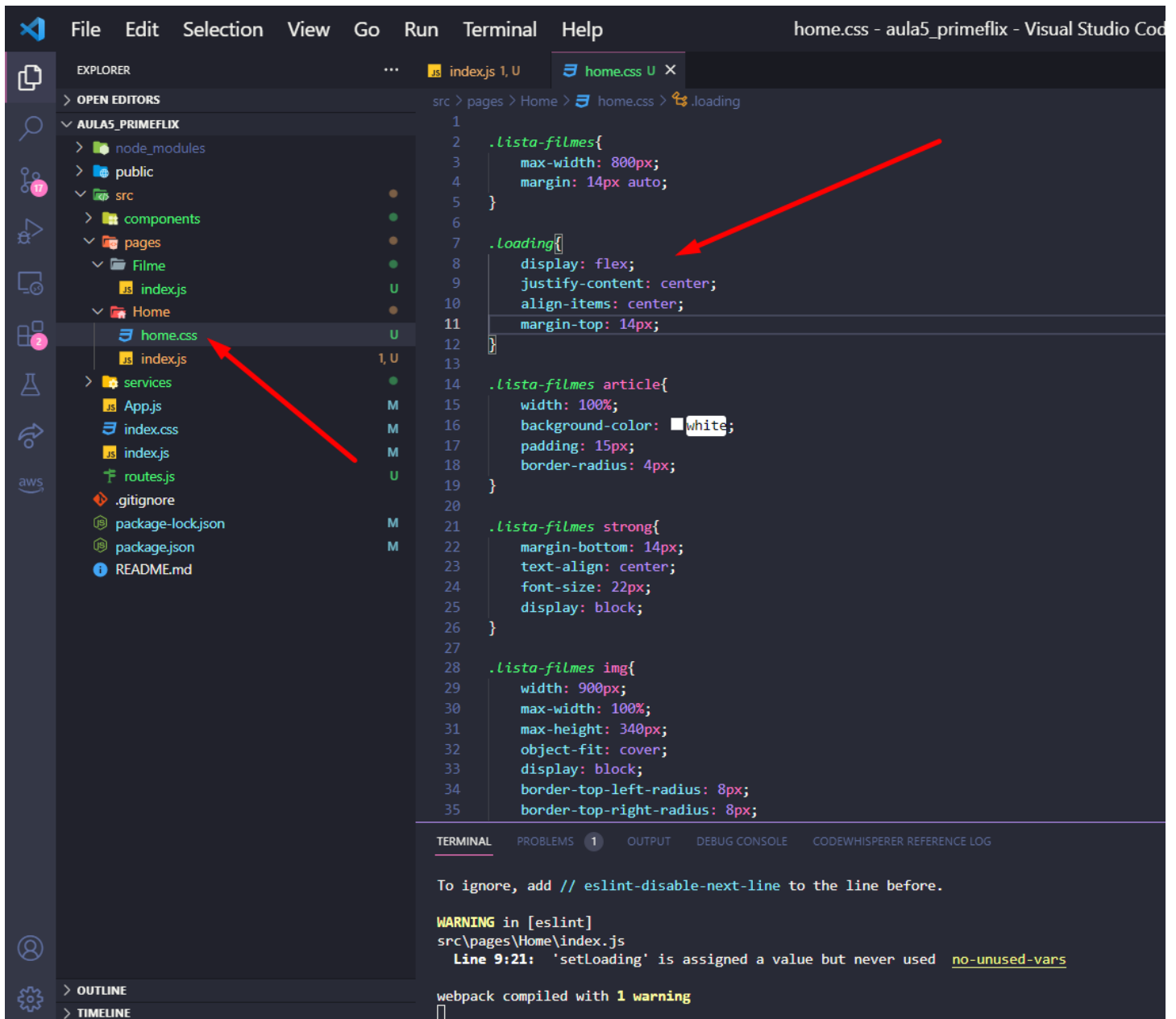
TERMINAL

To ignore, add // eslint-disable-next-line to the line before.

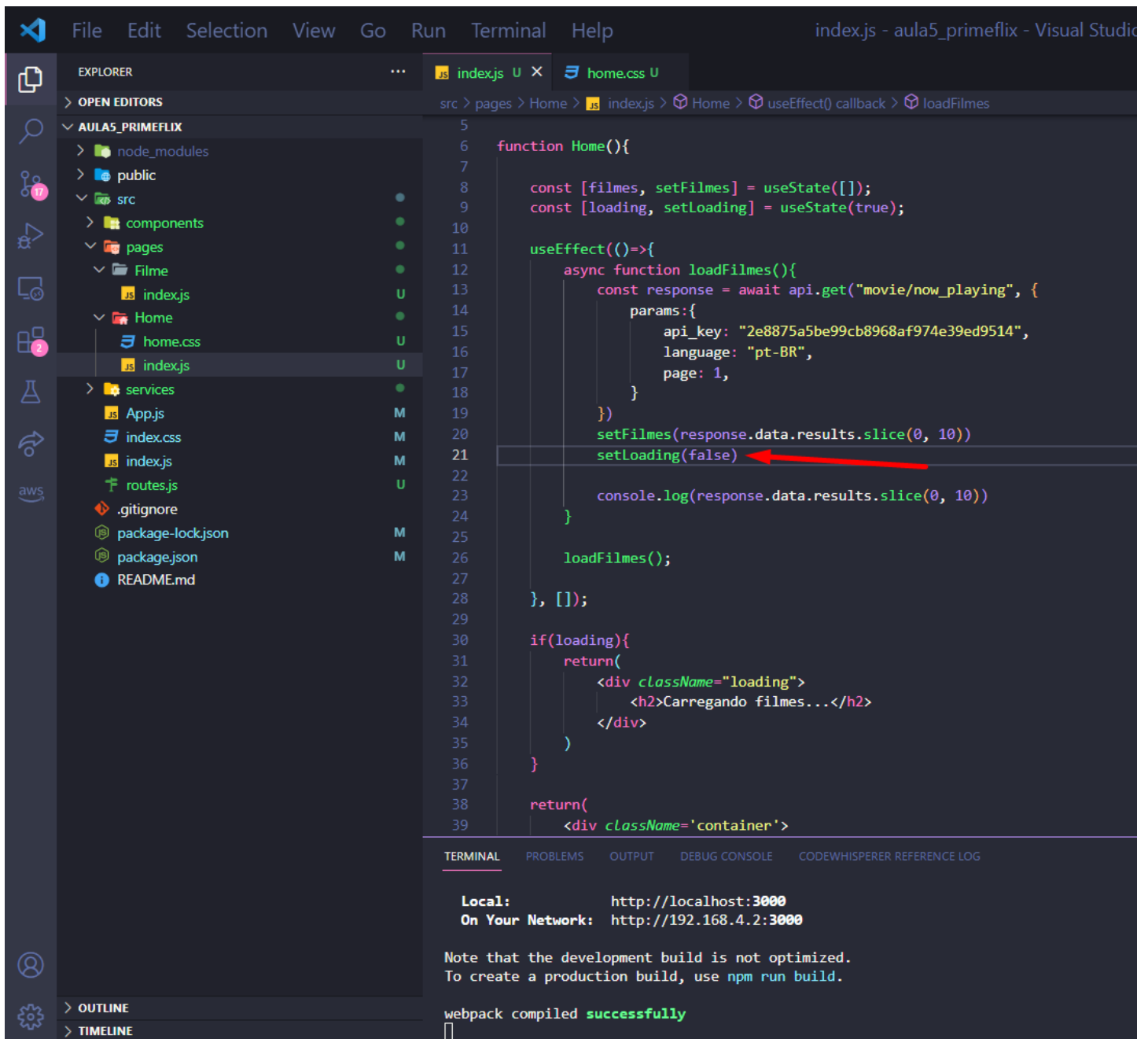
WARNING in [eslint]
src\pages\Home\index.js
Line 9:21: 'setLoading' is assigned a value but never used [no-unused-vars](#)

webpack compiled with 1 warning

03 – Vá até o arquivo home.css na pasta Home e vamos estilizar a classe loading.



04 – Se você observar nossa página, como o estado dela é true ela sempre irá exibir a mensagem “Carregando filme...”, então precisamos mudar esse estado para false assim que terminarmos de realizar a requisição da API e termos a lista de filmes na nossa aplicação. Volte ao index.js na pasta Home e adicione a mudança de estado ao final da função loadFilmes.



The screenshot shows the Visual Studio Code interface with the following details:

- Explorer Panel:** Displays the file structure of the project. The file `index.js` in the `Home` directory is selected.
- Editor Panel:** Shows the code for `index.js`. The code defines a `Home` function that uses `useState` to manage `filmes` and `loading` states. A red arrow points to the `setloading(false)` line at the end of the `loadFilmes` function.
- Terminal Panel:** Shows the output of the webpack compilation, indicating it was successful.

```
5
6 function Home(){
7
8   const [filmes, setFilmes] = useState([]);
9   const [loading, setloading] = useState(true);
10
11   useEffect(()=>{
12     async function loadFilmes(){
13       const response = await api.get("movie/now_playing", {
14         params:{
15           api_key: "2e8875a5be99cb8968af974e39ed9514",
16           language: "pt-BR",
17           page: 1,
18         }
19       });
20       setFilmes(response.data.results.slice(0, 10));
21       setloading(false);
22     }
23     console.log(response.data.results.slice(0, 10))
24   }
25   loadFilmes();
26 }, []);
27
28 if(loading){
29   return(
30     <div className="loading">
31       <h2>Carregando filmes...</h2>
32     </div>
33   )
34 }
35
36 return(
37   <div className='container'>
```

Local: http://localhost:3000
On Your Network: http://192.168.4.2:3000

Note that the development build is not optimized.
To create a production build, use `npm run build`.

webpack compiled successfully

05 – Vá até o navegador e no inspecionar (F12) na aba Network (Rede) mude o No throttling para Slow 3g, atualize sua página. Agora estamos simulando alguém com uma internet ruim, e enquanto a API não terminar seu processamento agora informamos o usuário que os filmes estão sendo carregados. Não esqueça que mudar novamente a velocidade do seu navegador.

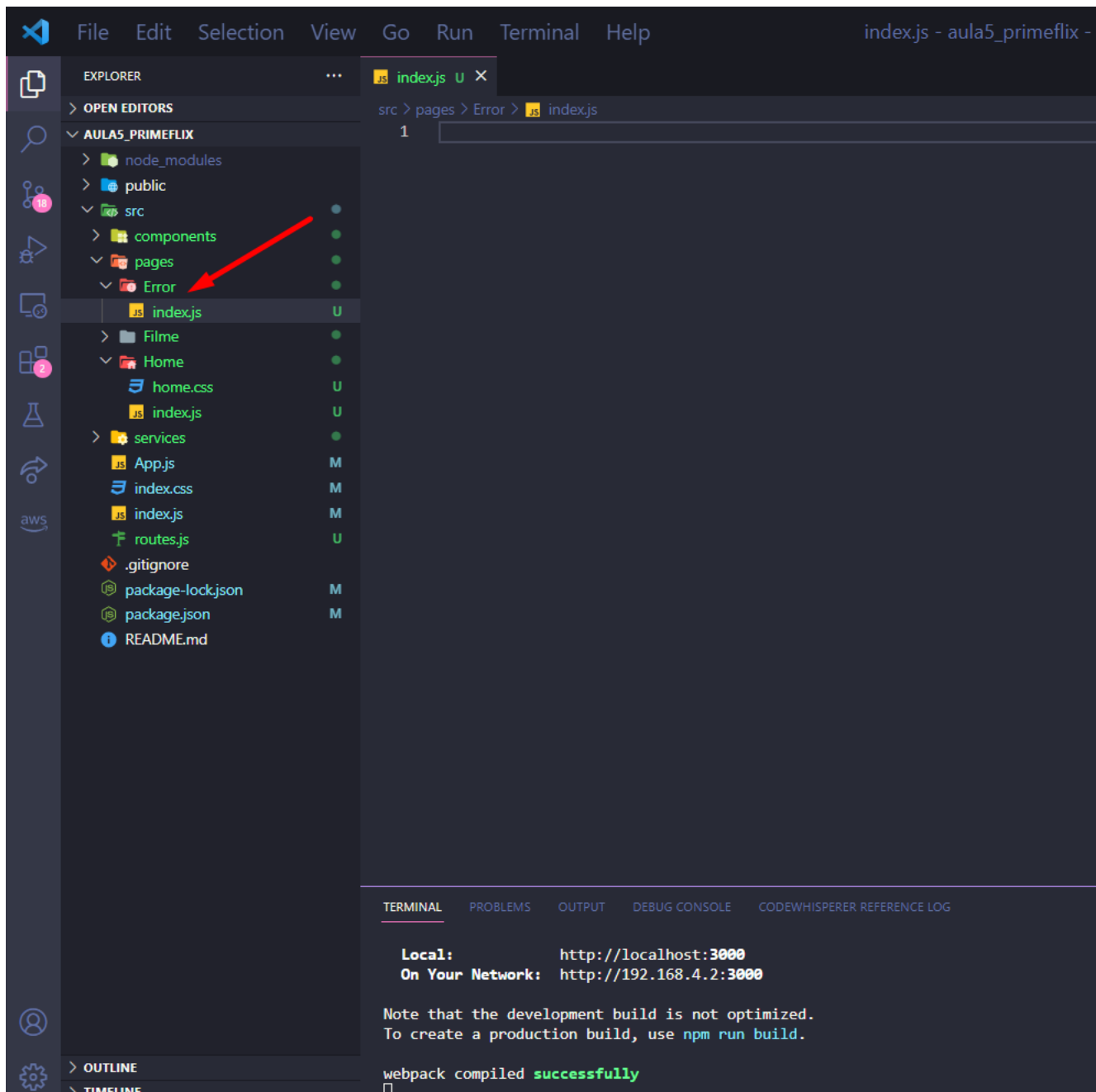
Com isso estamos melhorando a UX da página, como dependemos da API de outro serviço e não podemos prever se ela sempre irá responder rapidamente, nosso usuário não vai ficar dando reload na página em branco.

The screenshot shows a web browser at localhost:3000. The page has a black header with 'Prime Flix' and a button 'Meus Filmes'. Below the header, the text 'Carregando filmes...' is displayed. A red arrow points from this text to the Network tab in Chrome DevTools. The Network tab shows a list of requests, with 'bundle.js' highlighted. A red arrow points from the 'Slow 3G' throttling dropdown in the Network tab to the 'bundle.js' request. The 'bundle.js' request is shown as a green bar in the waterfall chart, indicating it is completed. The status bar at the bottom of the Network tab shows '5 requests', '601 B transferred', '2.1 MB resources', 'Finish: 4.09 s', 'DOMContentLoaded: 4.18 s', and 'Load: 4.27 s'.

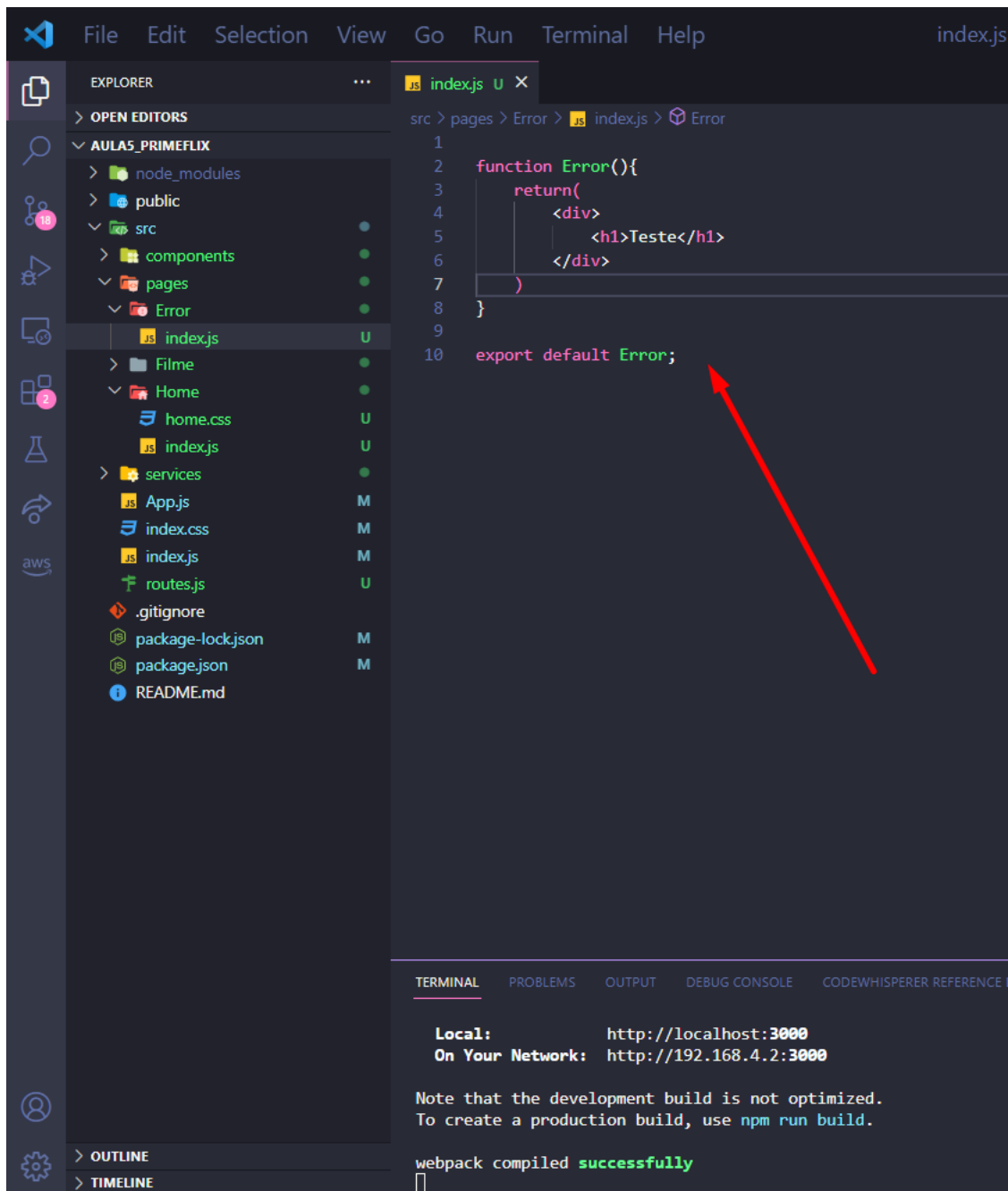
Name	Status	Type	Initiator	Size	Time	Waterfall
ws	(pending)	websoc...	bundle.js:source...	0 B	Pending	
localhost	304	docume...	Other	299 B	2.02 s	
bundle.js	304	script	(index)	302 B	2.03 s	
now_playing?api_key=2e8875a...	200	xhr	bundle.js:source...	0 B	Pending	
now_playing?api_key=2e8875a...	(pending)	xhr	bundle.js:source...	0 B	Pending	

Página de NOT FOUND

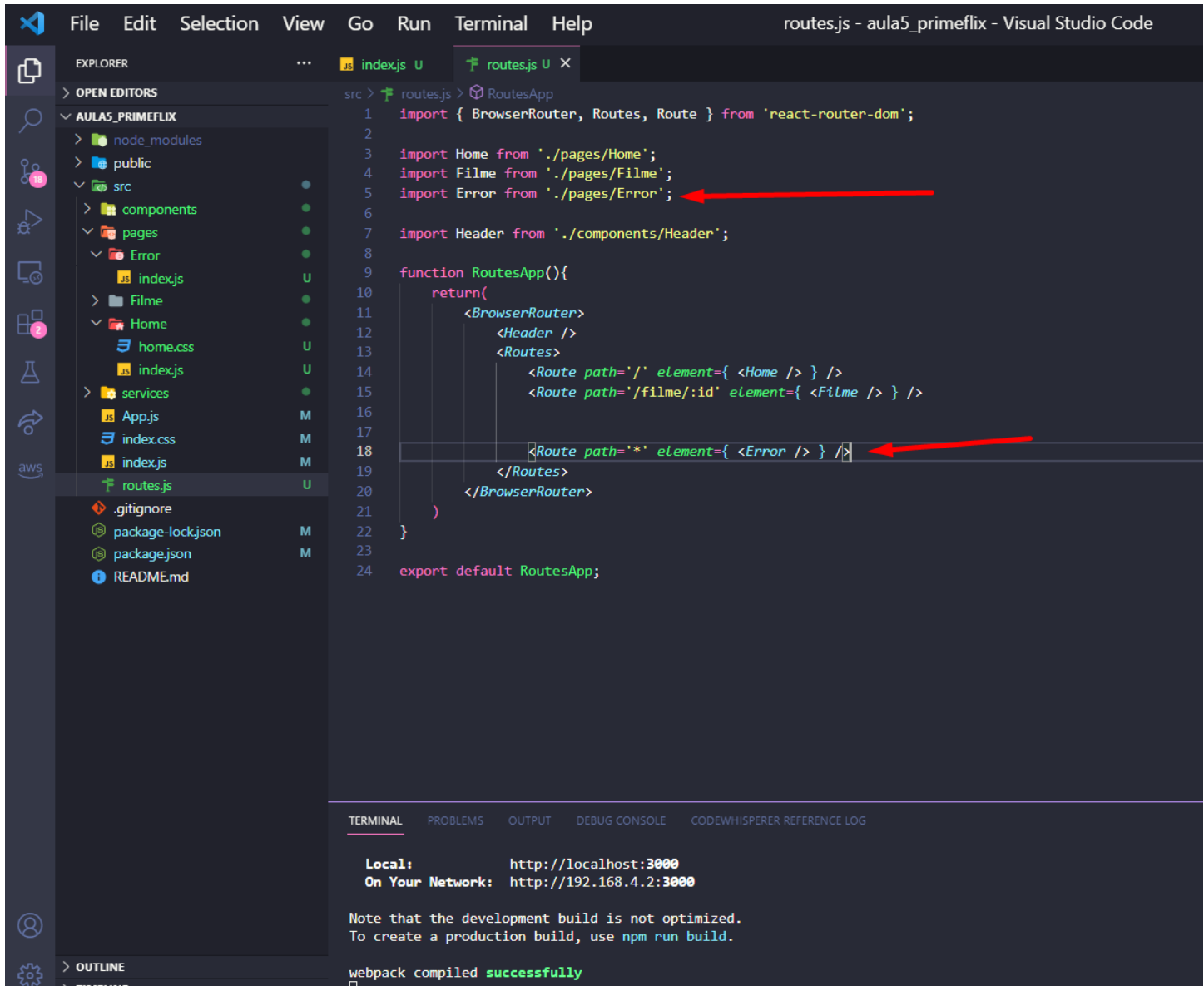
06 – Agora vamos implementar o caso onde se a página que o usuário estiver procurando não existir isso seja informado a ele. Crie uma nova pasta, dentro de pages, chamada Error e dentro um arquivo chamado index.js.



07 – Vamos fazer uma função padrão apenas para testar as rotas.



08 – Vá até o arquivo “routes.js” importe o Error indicando sua localização no projeto e dentro da função RoutesAPP, adicione ele com o path *, ou seja, qualquer página que não existir ele irá renderizar nosso componente de erro.



```
File Edit Selection View Go Run Terminal Help routes.js - aula5_primeflix - Visual Studio Code

EXPLORER
OPEN EDITORS
AULAS_PRIMEFLIX
  node_modules
  public
  src
    components
    pages
      Error
        index.js
      Filme
      Home
        home.css
        index.js
    services
      App.js
      index.css
      index.js
      routes.js
  .gitignore
  package-lock.json
  package.json
  README.md

src > routes.js > RoutesApp
1 import { BrowserRouter, Routes, Route } from 'react-router-dom';
2
3 import Home from './pages/Home';
4 import Filme from './pages/Filme';
5 import Error from './pages/Error';
6
7 import Header from './components/Header';
8
9 function RoutesApp(){
10   return(
11     <BrowserRouter>
12       <Header />
13       <Routes>
14         <Route path="/" element={ <Home /> } />
15         <Route path="/filme/:id" element={ <Filme /> } />
16         <Route path="*" element={ <Error /> } />
17       </Routes>
18     </BrowserRouter>
19   )
20 }
21
22 export default RoutesApp;

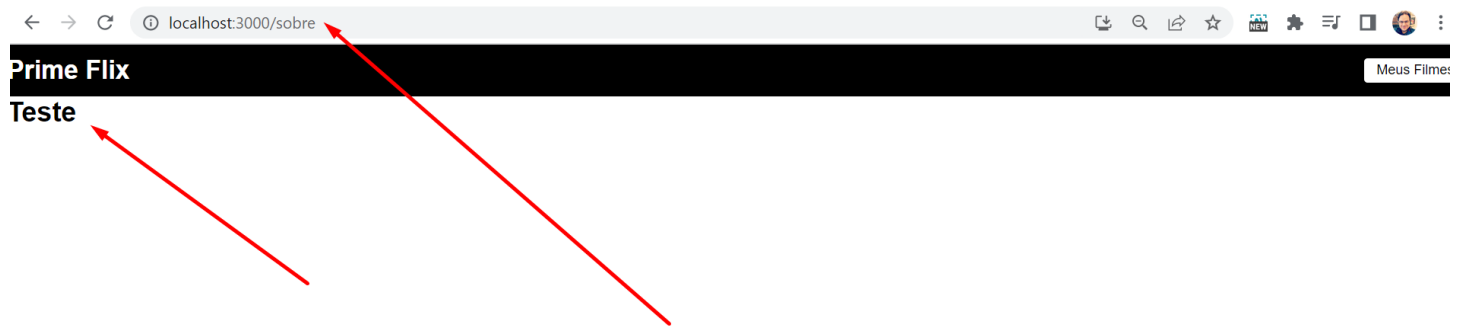
TERMINAL
PROBLEMS OUTPUT DEBUG CONSOLE CODEWHISPERER REFERENCE LOG

Local: http://localhost:3000
On Your Network: http://192.168.4.2:3000

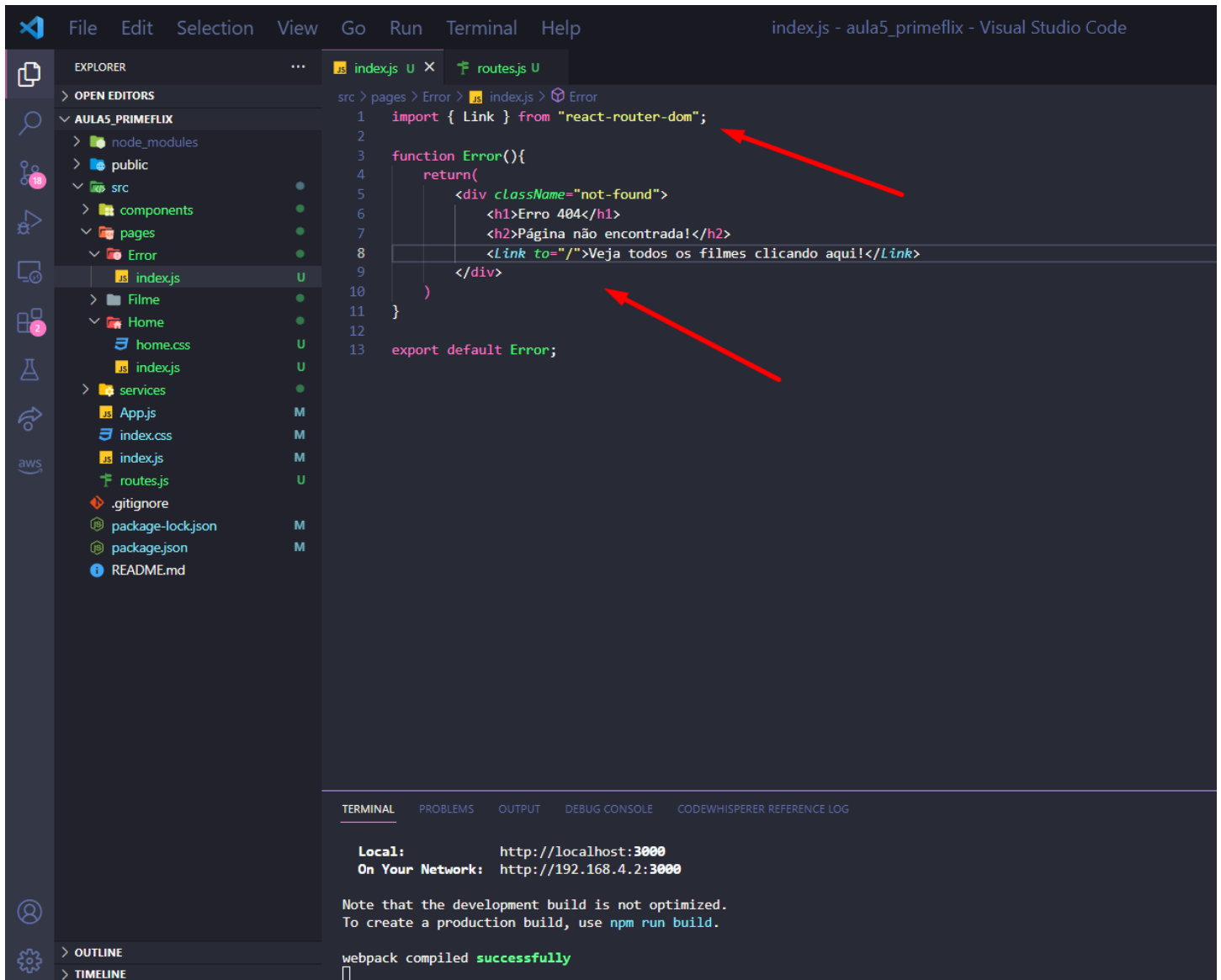
Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```

09 – Vamos testar, entre por exemplo na página sobre, que não existe, ele irá mostrar nossa div de teste.



10 – Volte para o index.js na pasta Error, vamos agora criar uma página de erro padrão e deixar um link para ajudar o usuário a se localizar e voltar para a Home (to="/").



The screenshot shows the Visual Studio Code interface with the Explorer, Editor, and Terminal panels. The Explorer panel on the left shows the project structure, with the file `index.js` in the `Error` folder selected. The Editor panel displays the code for `index.js`, which imports `Link` from `react-router-dom` and defines an `Error` component. The component returns a JSX element with a `div` containing an `h1`, an `h2`, and a `Link` pointing to the root path. The Terminal panel at the bottom shows the output of the development server, indicating that the application is running on `http://localhost:3000` and that webpack compiled successfully.

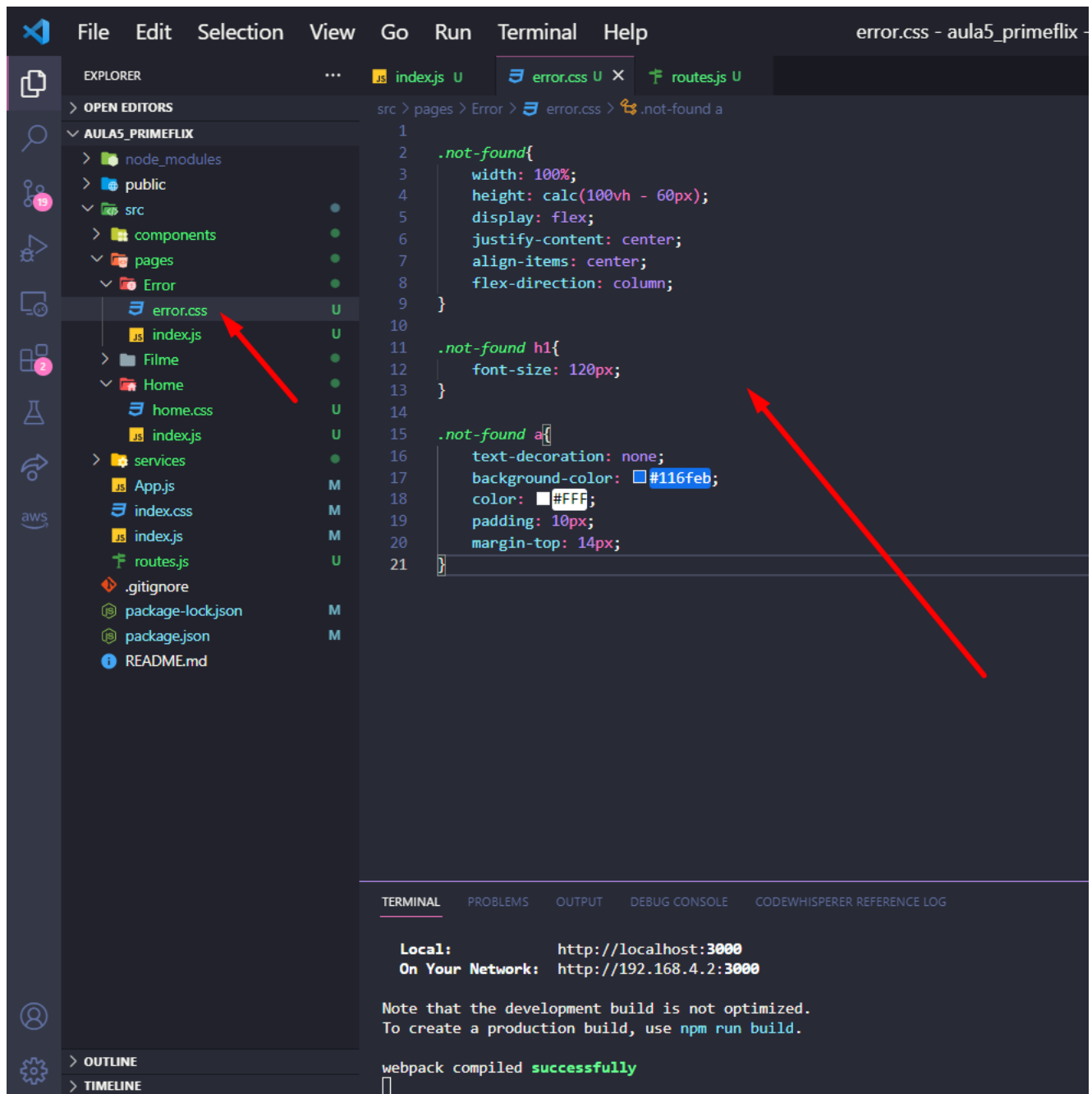
```
1 import { Link } from "react-router-dom";
2
3 function Error(){
4   return(
5     <div className="not-found">
6       <h1>Erro 404</h1>
7       <h2>Página não encontrada!</h2>
8       <Link to="/">Veja todos os filmes clicando aqui!</Link>
9     </div>
10  )
11 }
12
13 export default Error;
```

Local: http://localhost:3000
On Your Network: http://192.168.4.2:3000

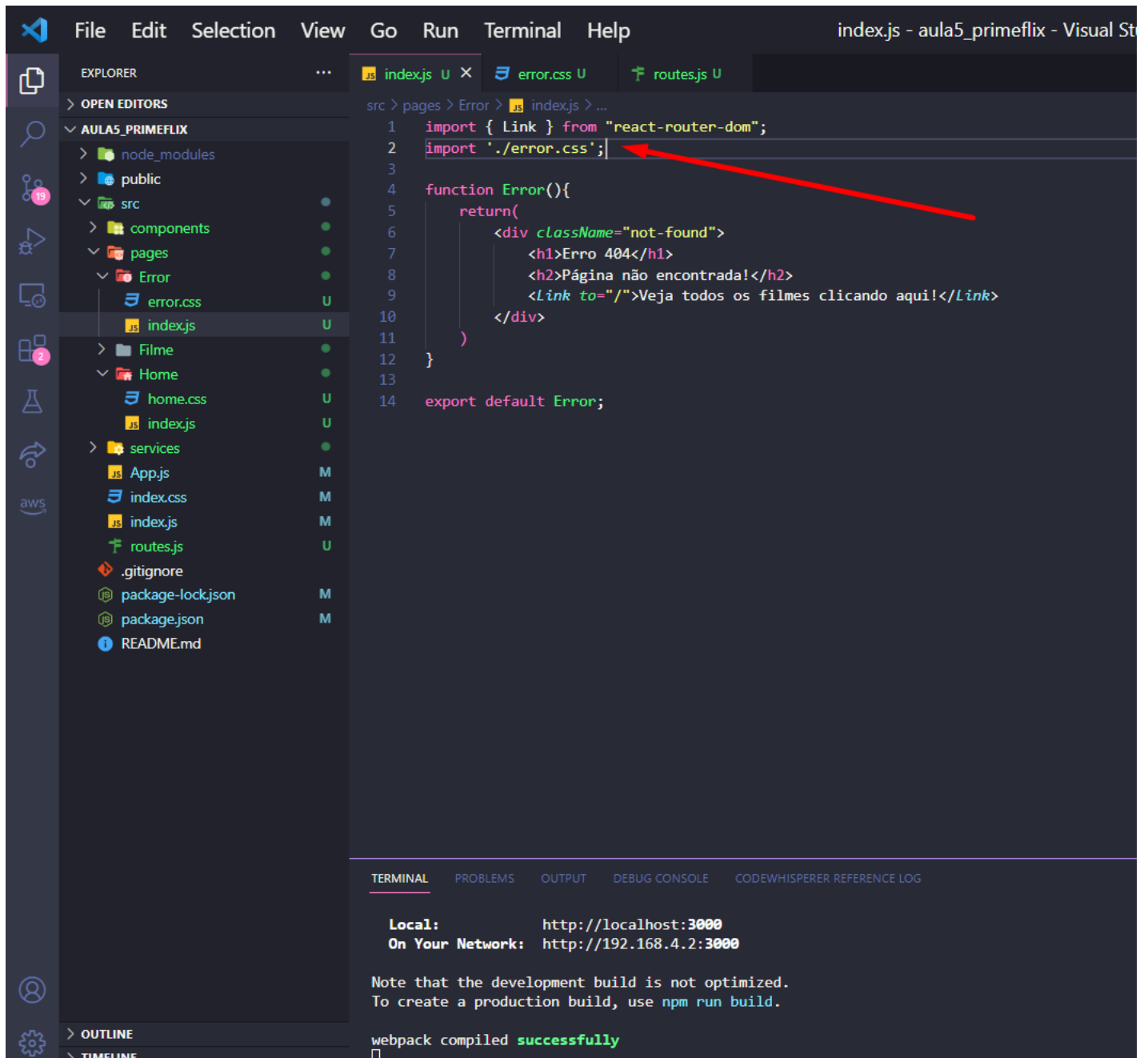
Note that the development build is not optimized.
To create a production build, use `npm run build`.

webpack compiled **successfully**

11 – Vamos agora estilizar nossa página de Not Found, crie um arquivo chamado error.css dentro da página Error, coloque o CSS abaixo como sugestão.



12 – Volte ao index.js dentro da pasta Error e import nosso arquivo error.css.



```
1 import { Link } from "react-router-dom";
2 import './error.css';
3
4 function Error(){
5   return(
6     <div className="not-found">
7       <h1>Erro 404</h1>
8       <h2>Página não encontrada!</h2>
9       <Link to="/">Veja todos os filmes clicando aqui!</Link>
10     </div>
11   )
12 }
13
14 export default Error;
```

Local: http://localhost:3000
On Your Network: http://192.168.4.2:3000

Note that the development build is not optimized.
To create a production build, use `npm run build`.

webpack compiled **successfully**

12 – Este será o resultado final da página de Not-Found.



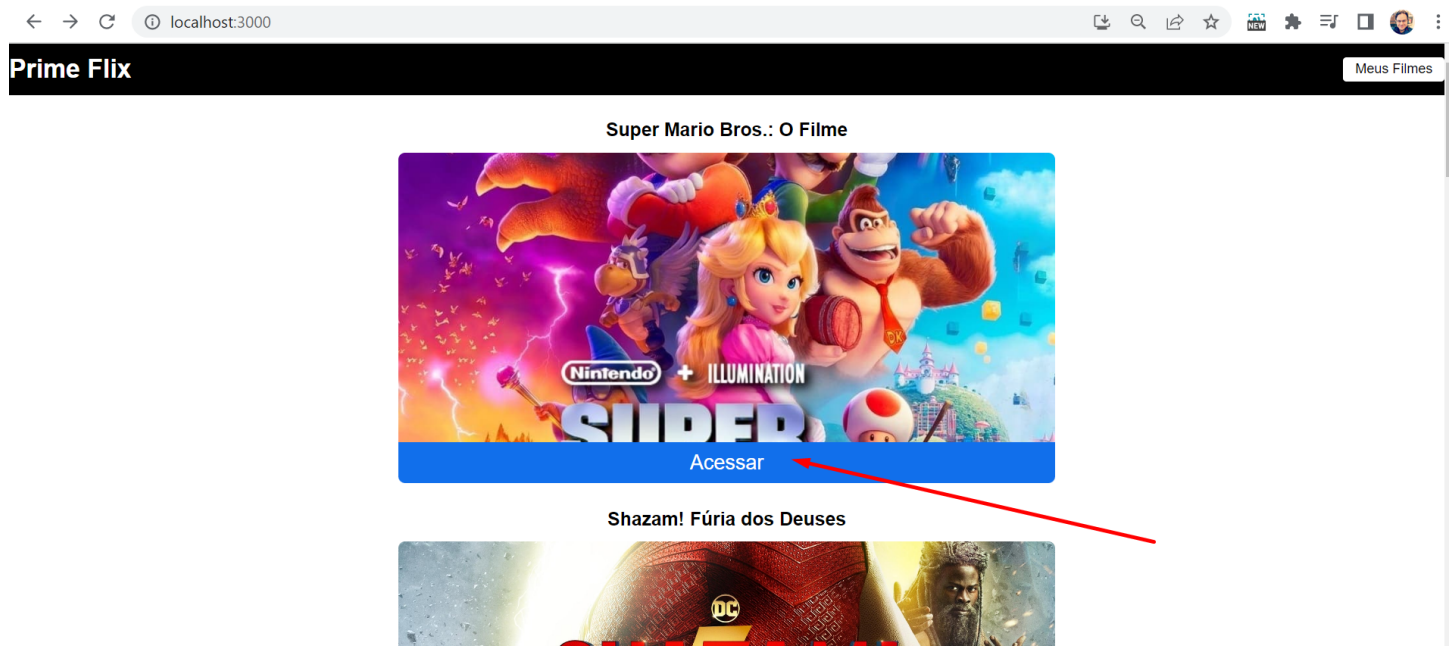
Erro 404

Página não encontrada!

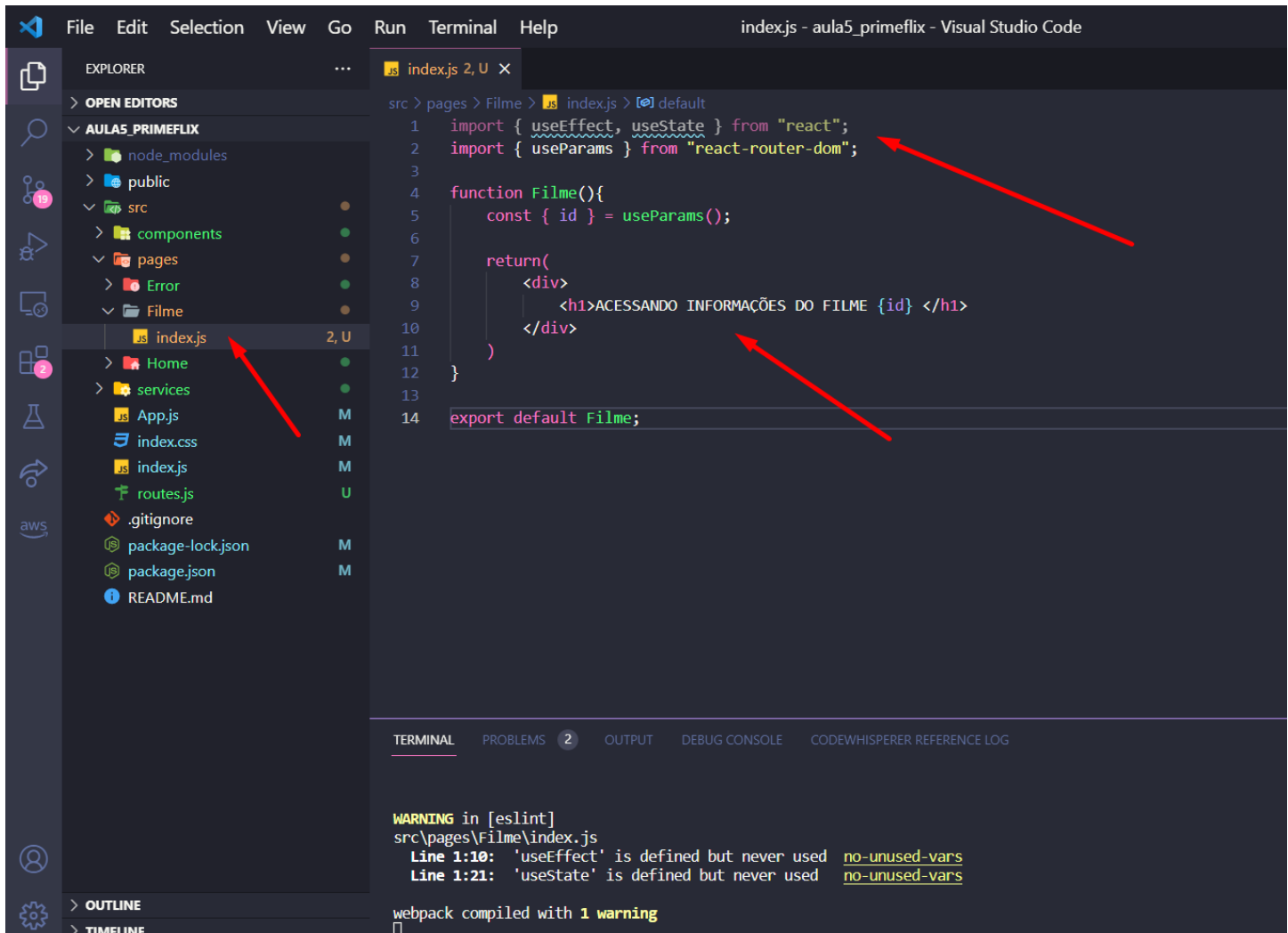
[Veja todos os filmes clicando aqui!](#)

Detalhando - Criando dinamicamente páginas para cada filme

13 – Atualmente em nossa aplicação, ao usuário clicar em acessar abaixo de cada filma da Home, uma nova página que ainda não exibe de fato os detalhes do filme que o usuário deseja ver. Iremos implementar isso agora!



14 – Sabemos que cada filme tem um id e estamos utilizando esse id para gerar rotas dinâmicas, continuando nessa lógica, vá até o index.js de filmes. Vamos importar o useEffect, useState e useParams. Por hora vamos ver se conseguimos acessar o id do filme nessa página.



```
File Edit Selection View Go Run Terminal Help index.js - aula5_primeflix - Visual Studio Code

EXPLORER
OPEN EDITORS
AULAS_PRIMEFLIX
  node_modules
  public
  src
    components
    pages
      Error
      Filme
        index.js 2, U
      Home
      services
        App.js
        index.css
        index.js
        routes.js
      .gitignore
      package-lock.json
      package.json
      README.md

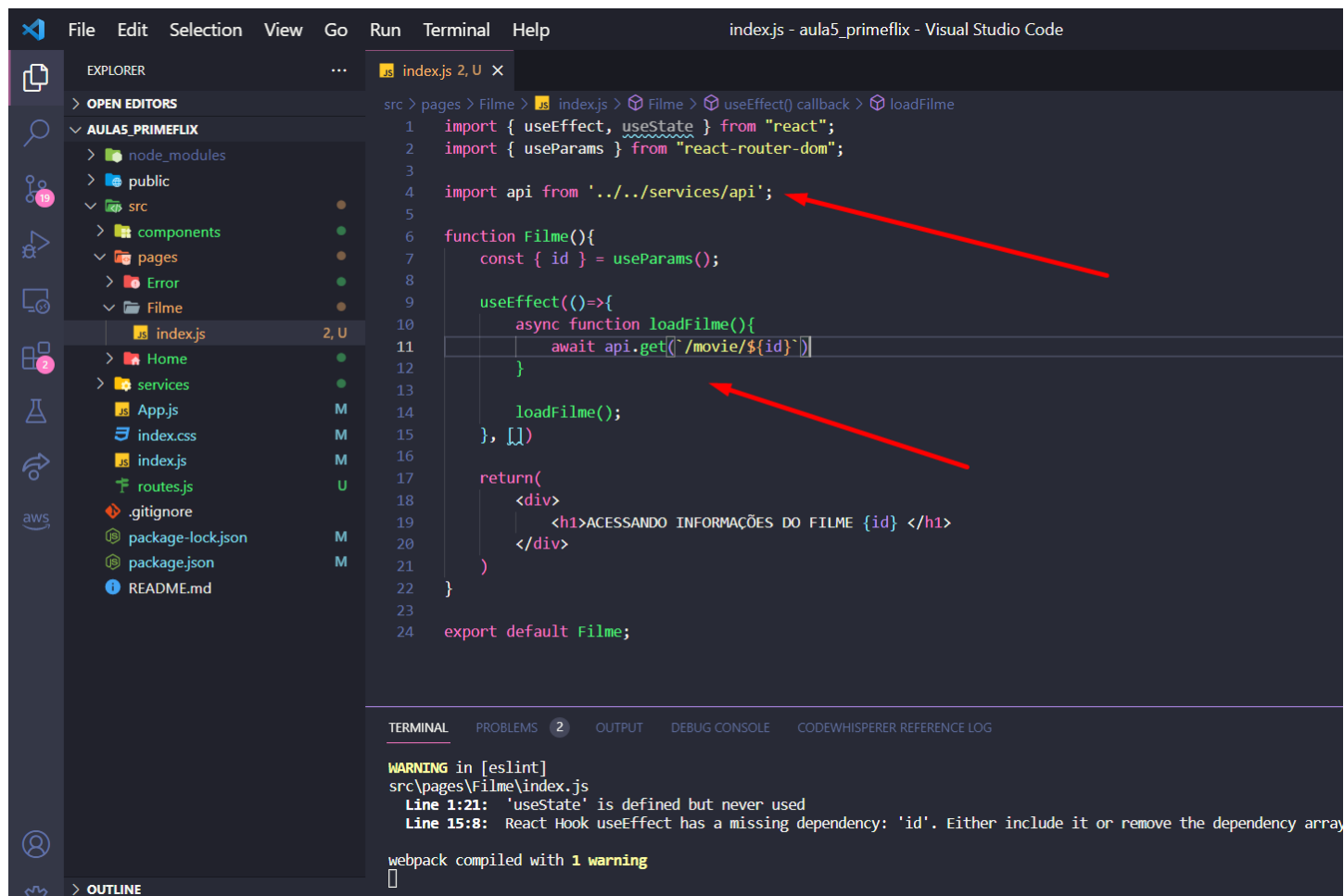
src > pages > Filme > index.js > default
1 import { useEffect, useState } from "react";
2 import { useParams } from "react-router-dom";
3
4 function Filme(){
5   const { id } = useParams();
6
7   return(
8     <div>
9       <h1>ACESSANDO INFORMAÇÕES DO FILME {id} </h1>
10     </div>
11   )
12 }
13
14 export default Filme;

TERMINAL
WARNING in [eslint]
src\pages\Filme\index.js
  Line 1:10: 'useEffect' is defined but never used  no-unused-vars
  Line 1:21: 'useState' is defined but never used    no-unused-vars
webpack compiled with 1 warning
```

15 – Veja que conseguimos sim acessar o id do filme na página, isso será base para consultarmos os detalhes do filme na API para exibir aqui.



16 – Precisamos requisitar para a API os detalhes do filme. Importe o service da API (linha 4), e vamos criar um `useEffect` dentro da função `Filme` que irá utilizar o `axis` e dar uma requisição GET no endpoint da API. Lembre-se que dentro do `get` utilizamos o template string, ou seja, não é aspas simples mas sim `` (shift + tecla ao lado da P).



```
File Edit Selection View Go Run Terminal Help index.js - aula5_primeflix - Visual Studio Code

EXPLORER
...
OPEN EDITORS
AULA5_PRIMEFLIX
  > node_modules
  > public
  > src
    > components
    > pages
    > Error
    > Filme
      index.js 2, U
  > Home
  > services
    App.js M
    index.css M
    index.js M
    routes.js U
  .gitignore
  package-lock.json M
  package.json M
  README.md

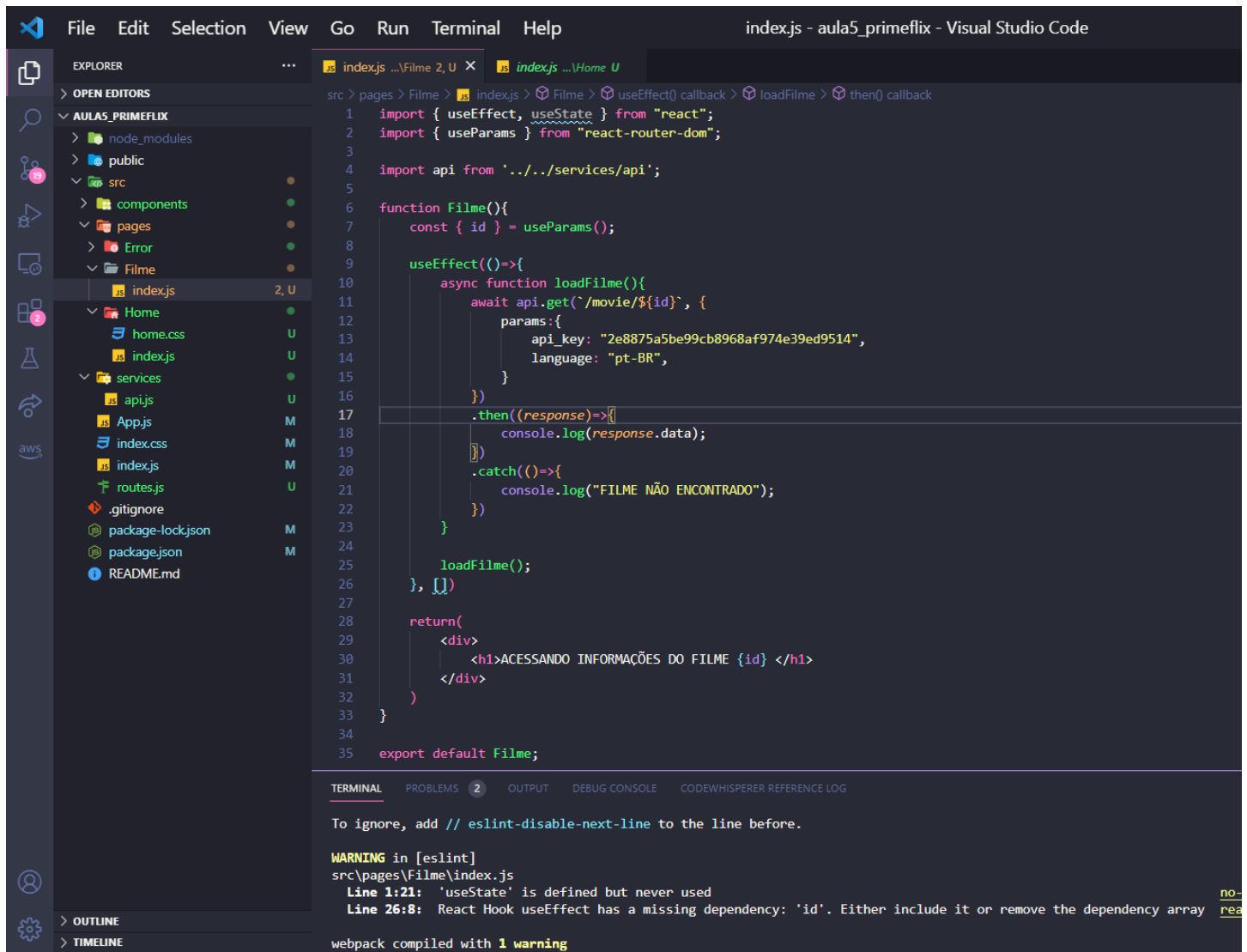
src > pages > Filme > index.js > Filme > useEffect() callback > loadFilme
1 import { useEffect, useState } from "react";
2 import { useParams } from "react-router-dom";
3
4 import api from '../services/api';
5
6 function Filme(){
7   const { id } = useParams();
8
9   useEffect(()=>{
10     async function loadFilme(){
11       await api.get(`/movie/${id}`);
12     }
13
14     loadFilme();
15   }, []);
16
17   return(
18     <div>
19       <h1>ACESSANDO INFORMAÇÕES DO FILME {id} </h1>
20     </div>
21   )
22 }
23
24 export default Filme;
```

TERMINAL PROBLEMS 2 OUTPUT DEBUG CONSOLE CODEWHISPERER REFERENCE LOG

WARNING in [eslint]
src\pages\Filme\index.js
Line 1:21: 'useState' is defined but never used
Line 15:8: React Hook useEffect has a missing dependency: 'id'. Either include it or remove the dependency array

webpack compiled with 1 warning

17 – Vamos montar os parametros dessa requisição GET passando a `api_key` e a `language`. Nós já fizemos a mesma requisição na Home caso queira consultar algo. Aproveitamos também para fazer um tratamento de erro usando o `then` e `catch`.



```
1 import { useEffect, useState } from "react";
2 import { useParams } from "react-router-dom";
3
4 import api from '../services/api';
5
6 function Filme(){
7   const { id } = useParams();
8
9   useEffect(()=>{
10     async function loadFilme(){
11       await api.get(`/movie/${id}`, {
12         params:{
13           api_key: "2e8875a5be99cb8968af974e39ed9514",
14           language: "pt-BR",
15         }
16       })
17     }
18     .then((response)=>[
19       console.log(response.data);
20     ])
21     .catch(()=>{
22       console.log("FILME NÃO ENCONTRADO");
23     })
24   })
25
26   loadFilme();
27 }
28
29 return(
30   <div>
31     <h1>ACESSANDO INFORMAÇÕES DO FILME {id} </h1>
32   </div>
33 )
34
35 export default Filme;
```

TERMINAL

To ignore, add // eslint-disable-next-line to the line before.

WARNING in [eslint]
src\pages\Filme\index.js
Line 1:21: 'useState' is defined but never used
Line 26:8: React Hook useEffect has a missing dependency: 'id'. Either include it or remove the dependency array

webpack compiled with 1 warning

18 – Vamos testar o código acima, por hora mandando para o console o resultado da api (response.data) e de fato conseguimos ver que os dados do filme desse id estão vindo corretos em português. Agora podemos utiliza-lo para montar os detalhes do filme.

The screenshot shows a web browser at the URL `localhost:3000/filme/502356`. The page has a header with "Prime Flix" and a button "Meus Filmes". Below the header, the text "ACESSANDO INFORMAÇÕES DO FILME 502356" is displayed. The React DevTools console is open, showing the JSON response from the API. A red arrow points from the text "ACESSANDO INFORMAÇÕES DO FILME 502356" to the console output.

```
react-dom.development.js:29840
Download the React DevTools for a better development experience: https://reactjs.org/link/react-devtools
index.js:18
{adult: false, backdrop_path: '/9n2tJBpLPbgR2ca05hS5CKXwP2c.jpg', belongs_to_c
ollection: null, budget: 100000000, genres: Array(5), ...}
  adult: false
  backdrop_path: "/9n2tJBpLPbgR2ca05hS5CKXwP2c.jpg"
  belongs_to_collection: null
  budget: 100000000
  genres: (5) [{...}, {...}, {...}, {...}, {...}]
  homepage: "https://cuevana3.it/pelicula/the-super-mario-bros-movie-2023/"
  id: 502356
  imdb_id: "tt6718170"
  original_language: "en"
  original_title: "The Super Mario Bros. Movie"
  overview: "Mario é um encanador junto com seu irmão Luigi. Um dia, eles vão ;
  popularity: 13700.561
  poster_path: "/i9XdxHsFrcqLkRWSF1coOHo4R39.jpg"
  production_companies: (3) [{...}, {...}, {...}]
```

Console What's New x Issues Network conditions

Highlights from the Chrome 112 update

CSS property documentation in the Styles pane

19 – Não queremos os dados do filme no console, por isso iremos manipular o resultado da requisição. Crie o useState de filme e loading. Altere o resultado de .then que agora irá passar via setFilme o resultado da API, também vamos parar de exibir a telha de loading de detalhes de filme. Falando nela, vamos criar um if para o loading e saber se os dados estão sendo carregados via api. Seu código deverá estar como o abaixo:

```
1 import { useEffect, useState } from "react";
2 import { useParams } from "react-router-dom";
3
4 import api from '../services/api';
5
6 function Filme(){
7   const { id } = useParams();
8   const [filme, setFilme] = useState({});
9   const [loading, setloading] = useState(true);
10
11   useEffect(()=>{
12     async function loadFilme(){
13       await api.get(`/movie/${id}`, {
14         params:{
15           api_key: "2e8875a5be99cb8968af974e39ed9514",
16           language: "pt-BR",
17         }
18       })
19       .then((response)=>{
20         setFilme(response.data);
21         setloading(false);
22       })
23       .catch(()=>{
24         console.log("FILME NÃO ENCONTRADO");
25       })
26     }
27
28     loadFilme();
29
30     return () => {
31       console.log("COMPONENTE FOI DESMONTADO")
32     }
33   }, [])
34
35   if(loading){
36     return(
37       <div className="filme-info">
38         <h1>Carregando detalhes do filme...</h1>
39       </div>
40     )
41   }
42
43   return(
44     <div>
45       <h1>ACESSANDO INFORMAÇÕES DO FILME {id}</h1>
46     </div>
47   )
48 }
49
50 export default Filme;
```

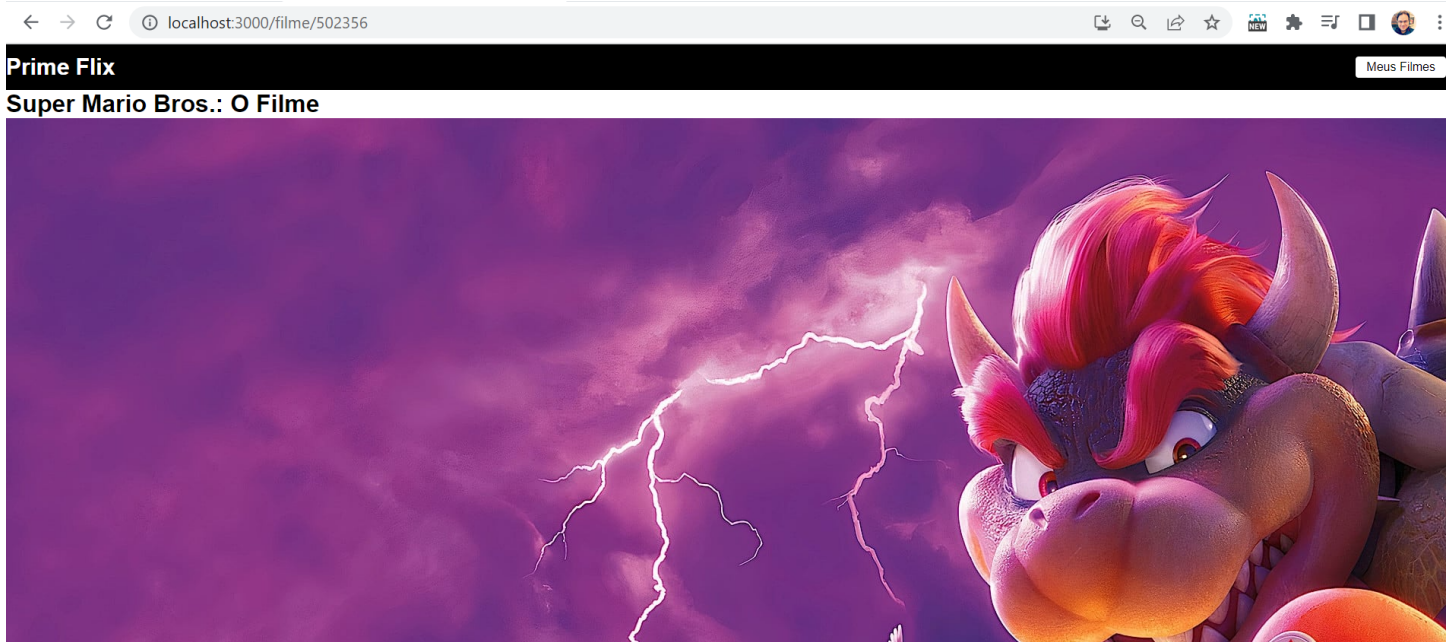
Line 27:8: React Hook useEffect has a missing dependency: 'id'. Either include it or remove the dependency array [react-hooks/exhaustive-deps](#)

20 – Agora vamos montar o que queremos exibir dos detalhes do filme. No último return, coloque o título do filme, puxe a imagem do poste, adicione uma Sinopse e a avaliação como mostrado abaixo.

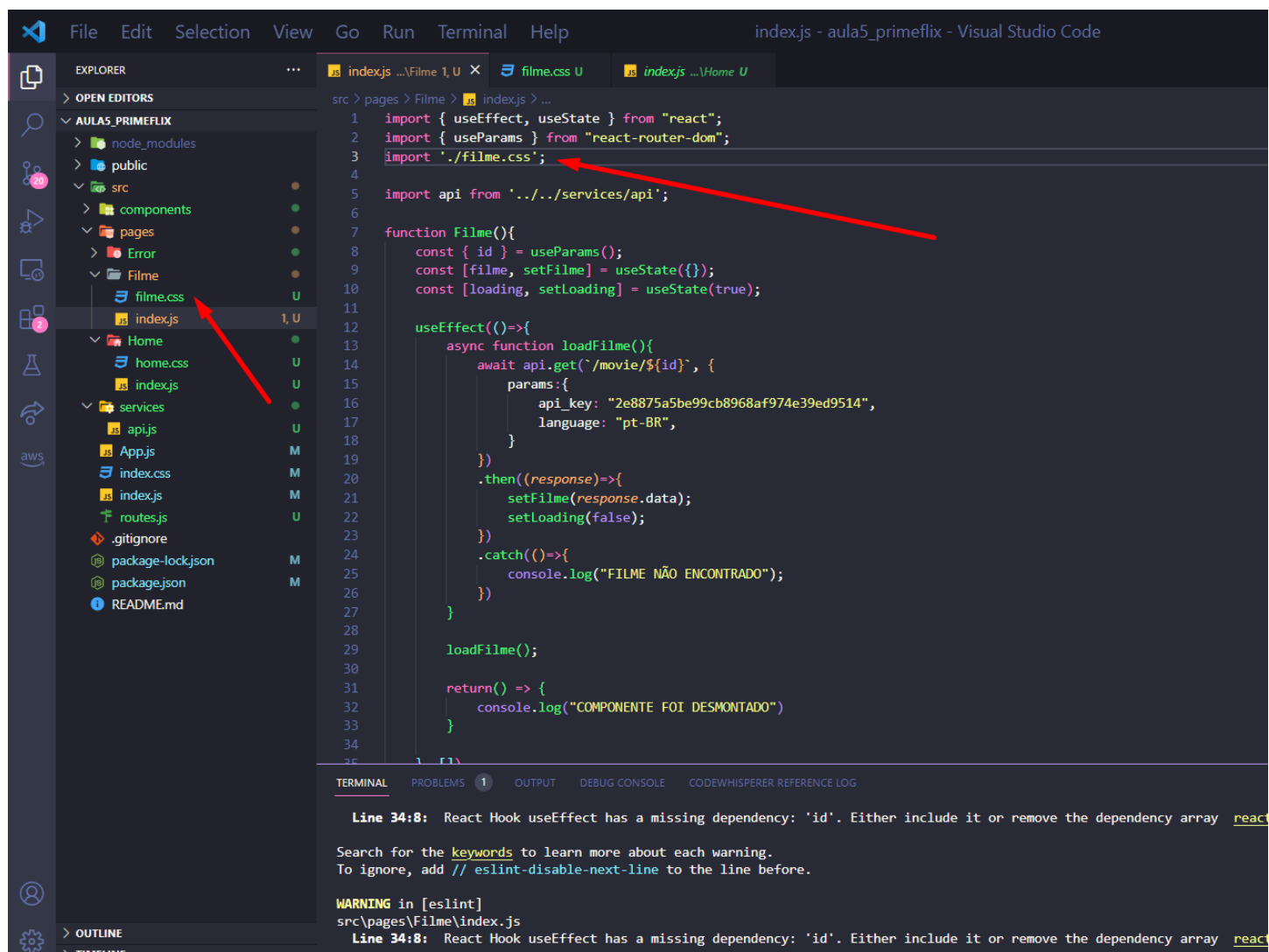
```
25      })
26    }
27
28    loadFilme();
29
30    return() => {
31      console.log("COMPONENTE FOI DESMONTADO")
32    }
33  }, [])
34
35  if (loading) {
36    return(
37      <div className="filme-info">
38        <h1>Carregando detalhes do filme...</h1>
39      </div>
40    )
41  }
42
43  return(
44    <div className="filme-info">
45      <h1>{filme.title}</h1>
46      <img src={`https://image.tmdb.org/t/p/original${filme.backdrop_path}`} alt={filme.title} />
47      <h3>Sinopse</h3>
48      <span>{filme.overview}</span>
49
50      <strong>Avaliação: {filme.vote_average} / 10 </strong>
51    </div>
52  )
53
54  }
55
56  export default Filme;
```

Line 34:8: React Hook useEffect has a missing dependency: 'id'. Either include it or remove the dependency array react-
Search for the keywords to learn more about each warning.
To ignore, add // eslint-disable-next-line to the line before.
WARNING in [eslint]
src\pages\Filme\index.js
Line 34:8: React Hook useEffect has a missing dependency: 'id'. Either include it or remove the dependency array react-

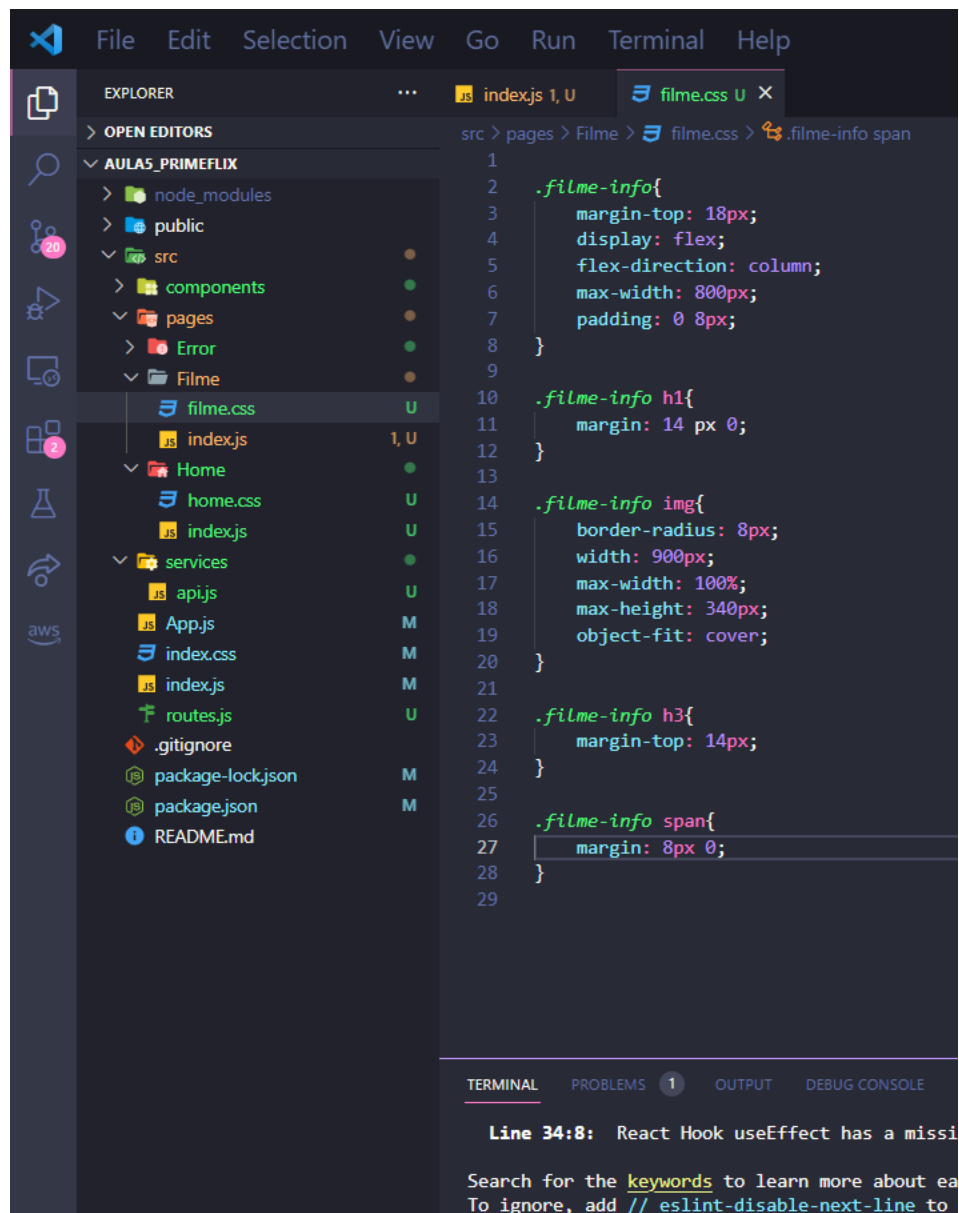
21 – Após a implementação acima realizada, esse será o resultado. Veja que as informações da API são exibidas na tela. Agora precisamos estilizar a página pois a imagem está muito grande e podemos melhorar a aparência geral.



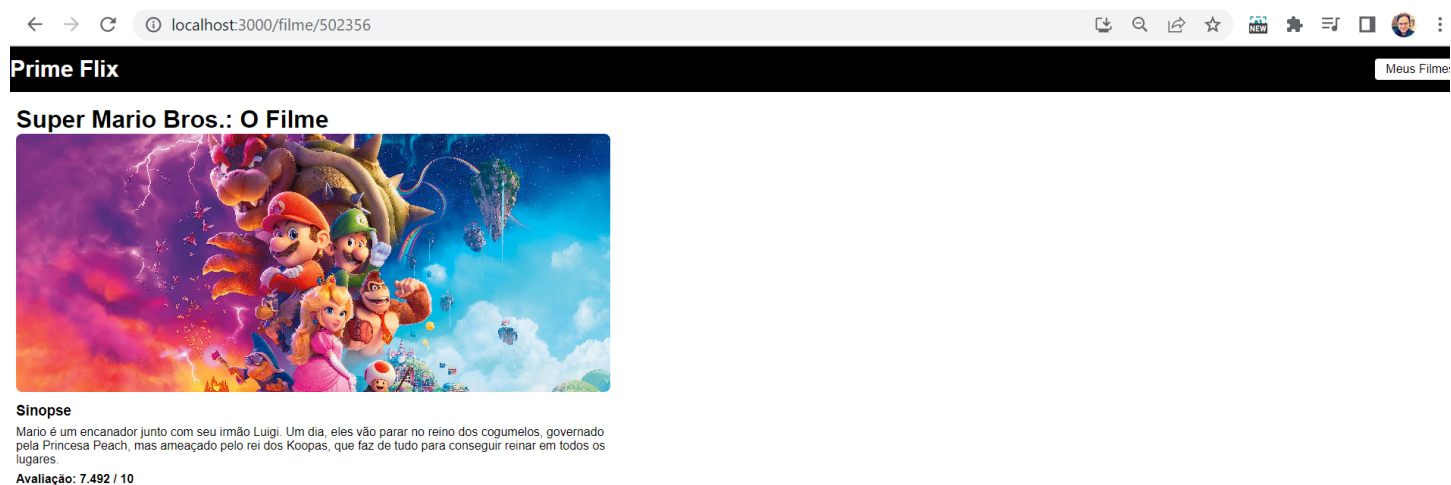
22 – Crie um novo arquivo chamado filme.css dentro da pasta Filme e já faça a importação desse arquivo dentro do index.js.



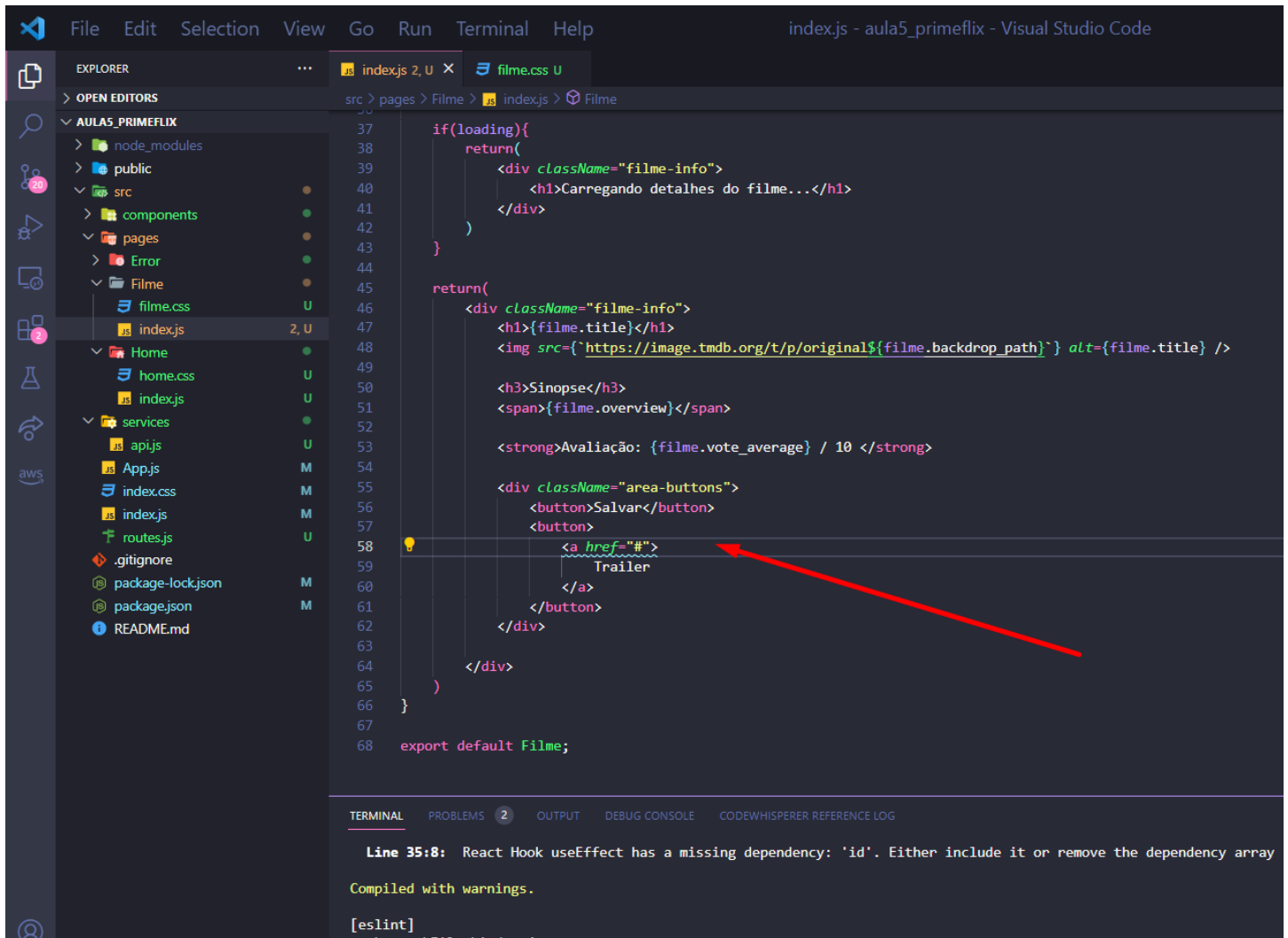
23 – Sugestão de estilização dos detalhes do filme. Fique à vontade para colocar seu estilo.



24 – Resultado do estilo feito anteriormente.



25 – Vamos também implementar 2 botões, um que irá permitir no futuro salvar em nossos favoritos e outro para que o usuário possa ver o trailer.



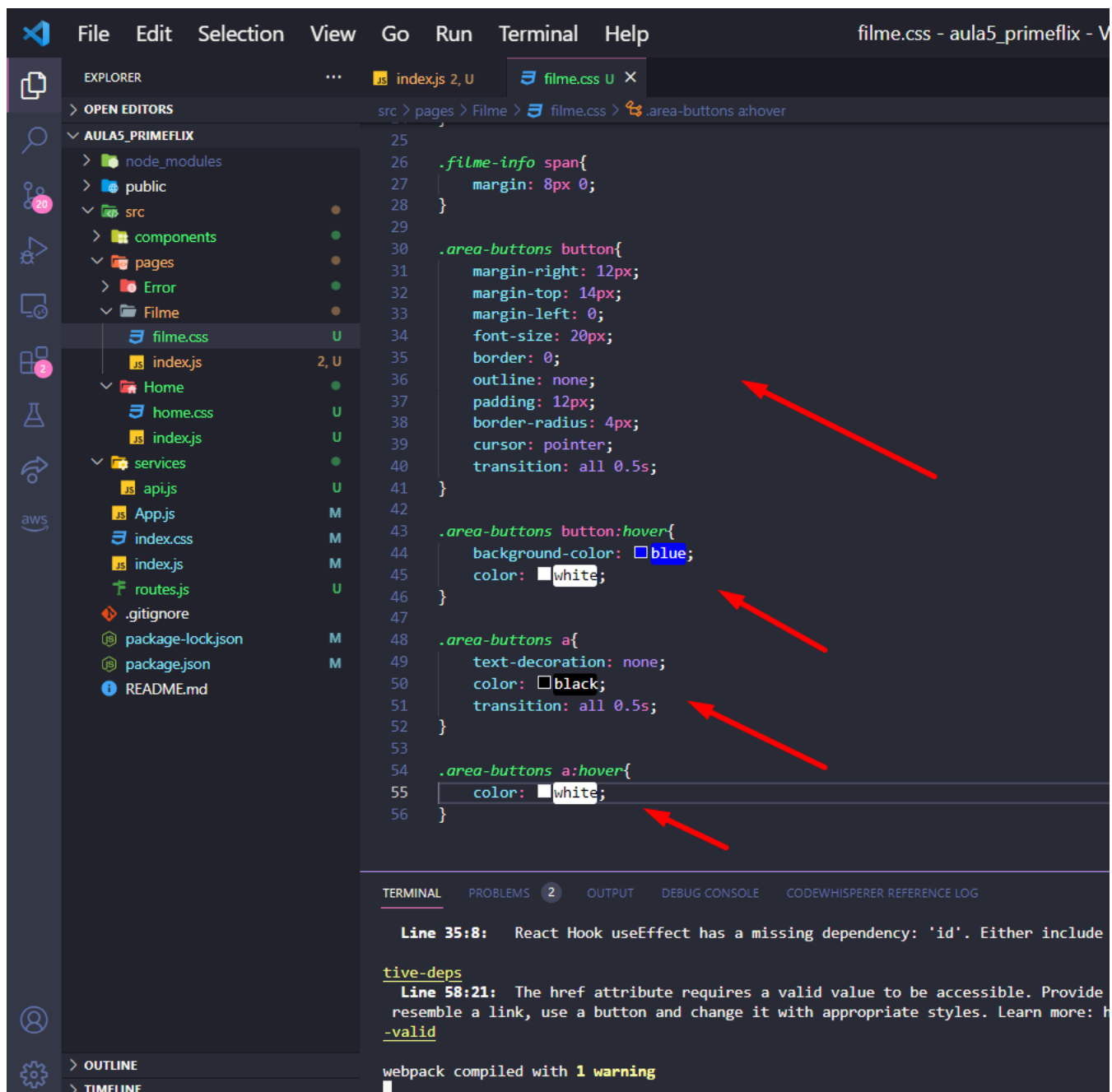
```
37   if (loading) {
38     return (
39       <div className="filme-info">
40         <h1>Carregando detalhes do filme...</h1>
41       </div>
42     )
43   }
44
45   return (
46     <div className="filme-info">
47       <h1>{filme.title}</h1>
48       <img src={`https://image.tmdb.org/t/p/original${filme.backdrop_path}`} alt={filme.title} />
49
50       <h3>Sinopse</h3>
51       <span>{filme.overview}</span>
52
53       <strong>Avaliação: {filme.vote_average} / 10 </strong>
54
55       <div className="area-buttons">
56         <button>Salvar</button>
57         <button>
58           <a href="#">
59             Trailer
60           </a>
61         </button>
62       </div>
63     </div>
64   )
65 }
66
67 export default Filme;
```

Line 35:8: React Hook useEffect has a missing dependency: 'id'. Either include it or remove the dependency array

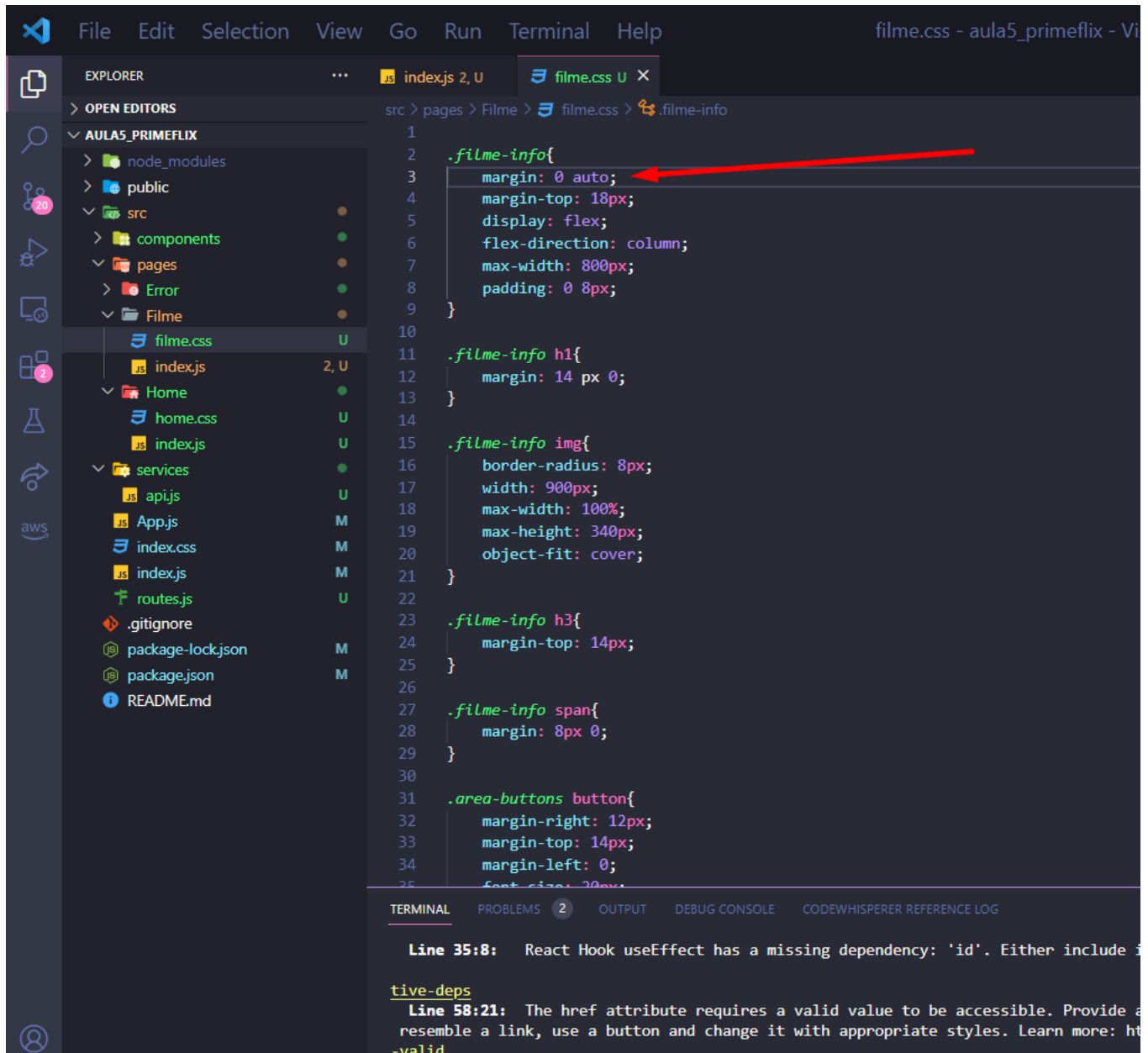
Compiled with warnings.

[eslint]

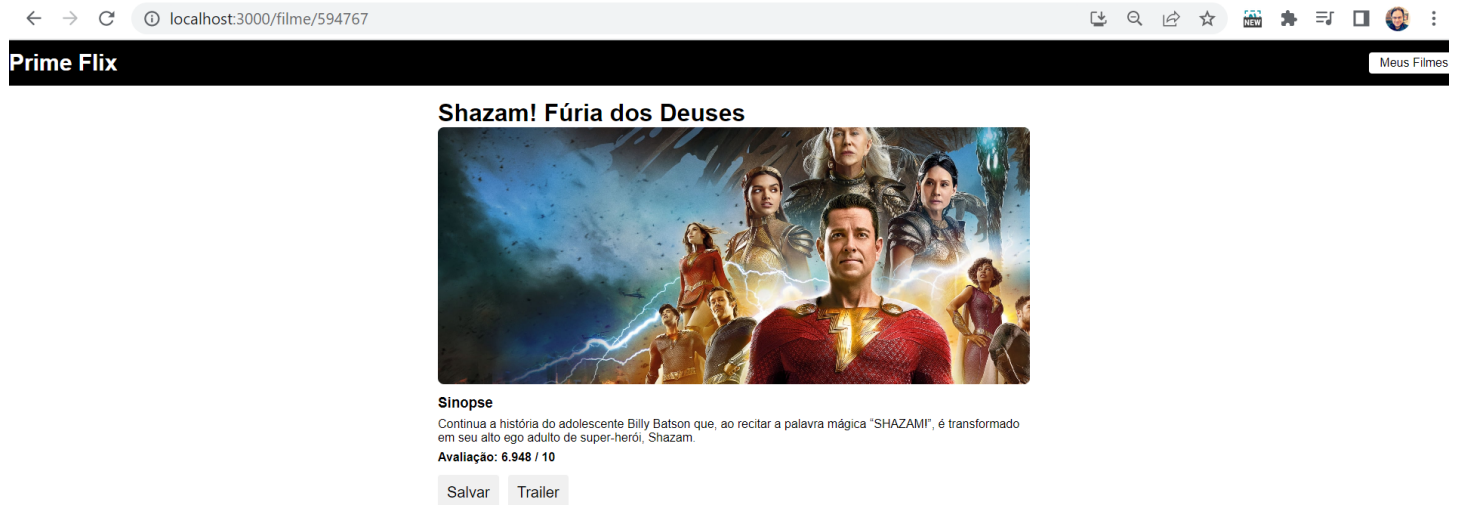
26 – Segue o estilo dos botões que acabamos de implementar.



27 – Uma melhoria está em deixar o conteúdo da página de detalhes todo centralizado, adicione o `margin: 0 auto;` dentro do `.filme-info`.

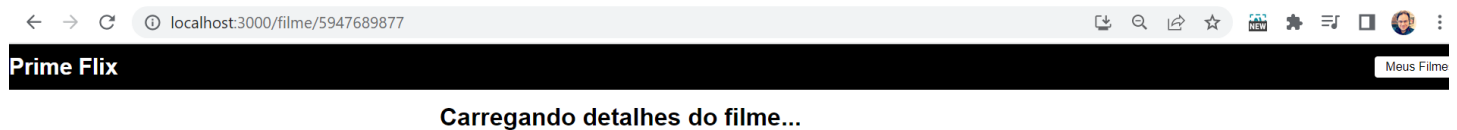


28 – Esse será o resultado da nossa página de detalhes que é gerada dinamicamente a partir do resultado da API, isso quer dizer que mesmo daqui 6 meses a página terá o mesmo comportamento porém com os filmes atuais.



Utilizando o hook useNavigate

29 – Temos atualmente um problema, ao usuário cair em um filme que não existe, essa tela de Carregando detalhes ficará sendo exibida para sempre. Vamos aprender agora um novo componente chamado Navigate.



30 – Faça a importação do `useNavigate` (linha 2), crie uma `const` chamada `navigate` para utilizarmos o `useNavigate`. E lá onde pegamos um possível erro no `.catch` chame o `navigate` e utilizando a propriedade do `replace` ele irá trocar a rota atual errado que o usuário digitou e enviar o usuário direto para a home. Faça um teste, digite um id que não existe e verifique se você será enviado para home.

```
1 import { useEffect, useState } from "react";
2 import { useParams, useNavigate } from "react-router-dom";
3 import './filme.css';
4
5 import api from '../services/api';
6
7 function Filme(){
8   const { id } = useParams();
9   const navigate = useNavigate();
10
11   const [filme, setFilme] = useState({});
12   const [loading, setLoading] = useState(true);
13
14   useEffect(()=>{
15     async function loadFilme(){
16       await api.get(`/movie/${id}`, {
17         params:{
18           api_key: "2e8875a5be99cb8968af974e39ed9514",
19           language: "pt-BR",
20         }
21       })
22       .then((response)=>{
23         setFilme(response.data);
24         setLoading(false);
25       })
26       .catch(()=>{
27         console.log("FILME NÃO ENCONTRADO");
28         navigate("/", { replace: true });
29         return;
30       })
31     }
32
33     loadFilme();
34
35     return () => {
36
37     }
38   })
39 }
```

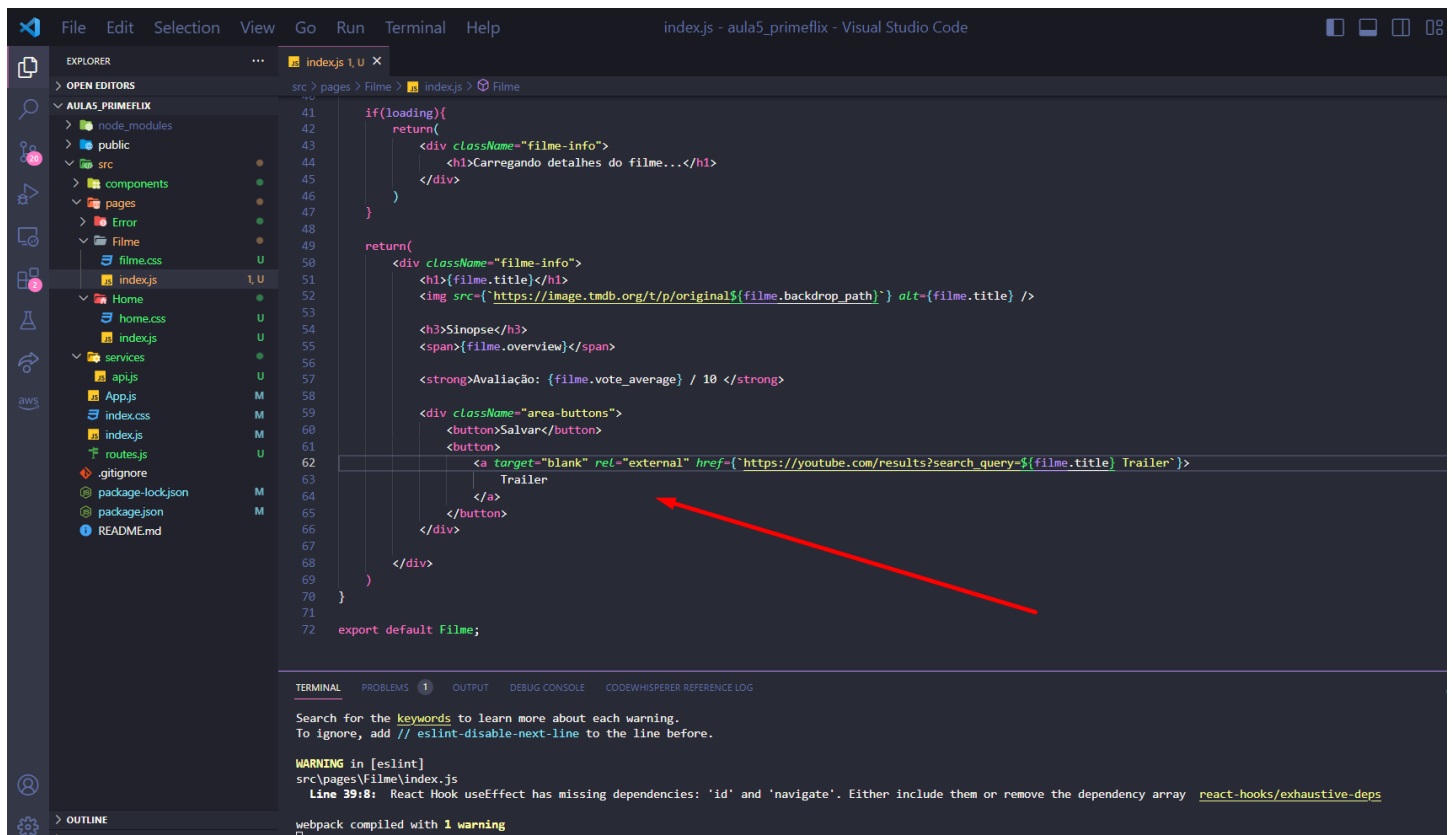
Line 39:8: React Hook `useEffect` has missing dependencies: `'id'` and `'navigate'`. Either include them or [use an empty array \[\]](#) to tell it that it does not depend on anything.

Line 62:21: The href attribute requires a valid value to be accessible. Provide a valid, navigable address that is either an absolute URL or a path that can be resolved to an absolute URL. For example, `"/foo/bar"` is a valid path, but `"/foo/bar"` is not. Learn more: <https://github.com/jsx-eslint/eslint-plugin-react/blob/master/docs/rules/valid-link-prop.js>

webpack compiled with 1 warning

Exibindo o trailer no youtube

29 – Como não temos possibilidade de armazenar os trailers do filme na aplicação, faremos com que toda vez que o usuário clique no botão Trailer uma consulta no youtube conteção com o Titulo do Filme, informação essa vindo da API, e a palavra trailer. Faça a modificação conforme abaixo é indicado:



```
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
export default Filme;

if (loading) {
  return (
    <div className="filme-info">
      <h1>Carregando detalhes do filme...</h1>
    </div>
  )
}

return (
  <div className="filme-info">
    <h1>{filme.title}</h1>
    <img src={`https://image.tmdb.org/t/p/original${filme.backdrop_path}`} alt={filme.title} />
    <h3>Sinopse</h3>
    <span>{filme.overview}</span>
    <strong>Avaliação: {filme.vote_average} / 10 </strong>
    <div className="area-buttons">
      <button>Salvar</button>
      <button>
        <a target="_blank" rel="external" href={`https://youtube.com/results?search_query=${filme.title} Trailer`} >
          Trailer
        </a>
      </button>
    </div>
  </div>
)
}
```

TERMINAL

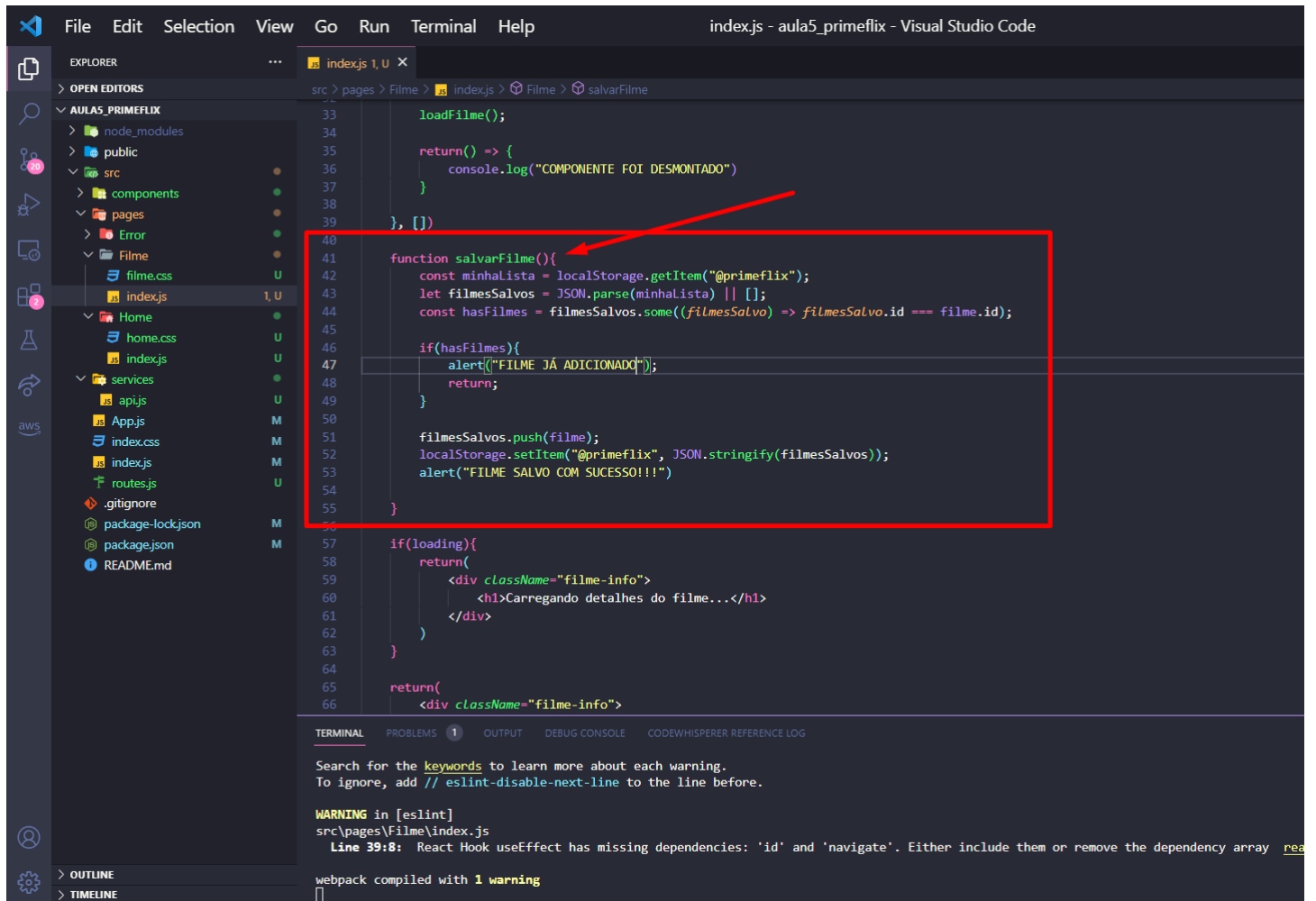
Search for the **keywords** to learn more about each warning.
To ignore, add `// eslint-disable-next-line` to the line before.

WARNING in [eslint]
src\pages\Filme\index.js
Line 39:8: React Hook useEffect has missing dependencies: 'id' and 'navigate'. Either include them or remove the dependency array [react-hooks/exhaustive-deps](#)

webpack compiled with 1 warning

Salvando meus filmes favoritos

30 – Iremos agora criar uma função chamada salvarFilme que utilizando o localStorage irá salvar localmente nosso filme. Também iremos criar uma lógica caso o filme já tenha sido adicionado anteriormente. Implemente a função abaixo:



```
File Edit Selection View Go Run Terminal Help index.js - aula5_primeflix - Visual Studio Code

EXPLORER
OPEN EDITORS
AULA5_PRIMEFLIX
  node_modules
  public
  src
    components
    pages
    Error
    Filme
      filme.css
      index.js
    Home
      home.css
      index.js
    services
      api.js
      App.js
      index.css
      index.js
      routes.js
  .gitignore
  package-lock.json
  package.json
  README.md

src > pages > Filme > index.js > Filme > salvarFilme

33 loadFilme();
34
35 return() => {
36   console.log("COMPONENTE FOI DESMONTADO")
37 }
38
39 }, [])
40
41 function salvarFilme(){
42   const minhaLista = localStorage.getItem("@primeflix");
43   let filmesSalvos = JSON.parse(minhaLista) || [];
44   const hasFilmes = filmesSalvos.some((filmeSalvo) => filmeSalvo.id === filme.id);
45
46   if(hasFilmes){
47     alert("FILME JÁ ADICIONADO");
48     return;
49   }
50
51   filmesSalvos.push(filme);
52   localStorage.setItem("@primeflix", JSON.stringify(filmesSalvos));
53   alert("FILME SALVO COM SUCESSO!!!")
54 }
55
56
57 if(loading){
58   return(
59     <div className="filme-info">
60       <h1>Carregando detalhes do filme...</h1>
61     </div>
62   )
63 }
64
65 return(
66   <div className="filme-info">
```

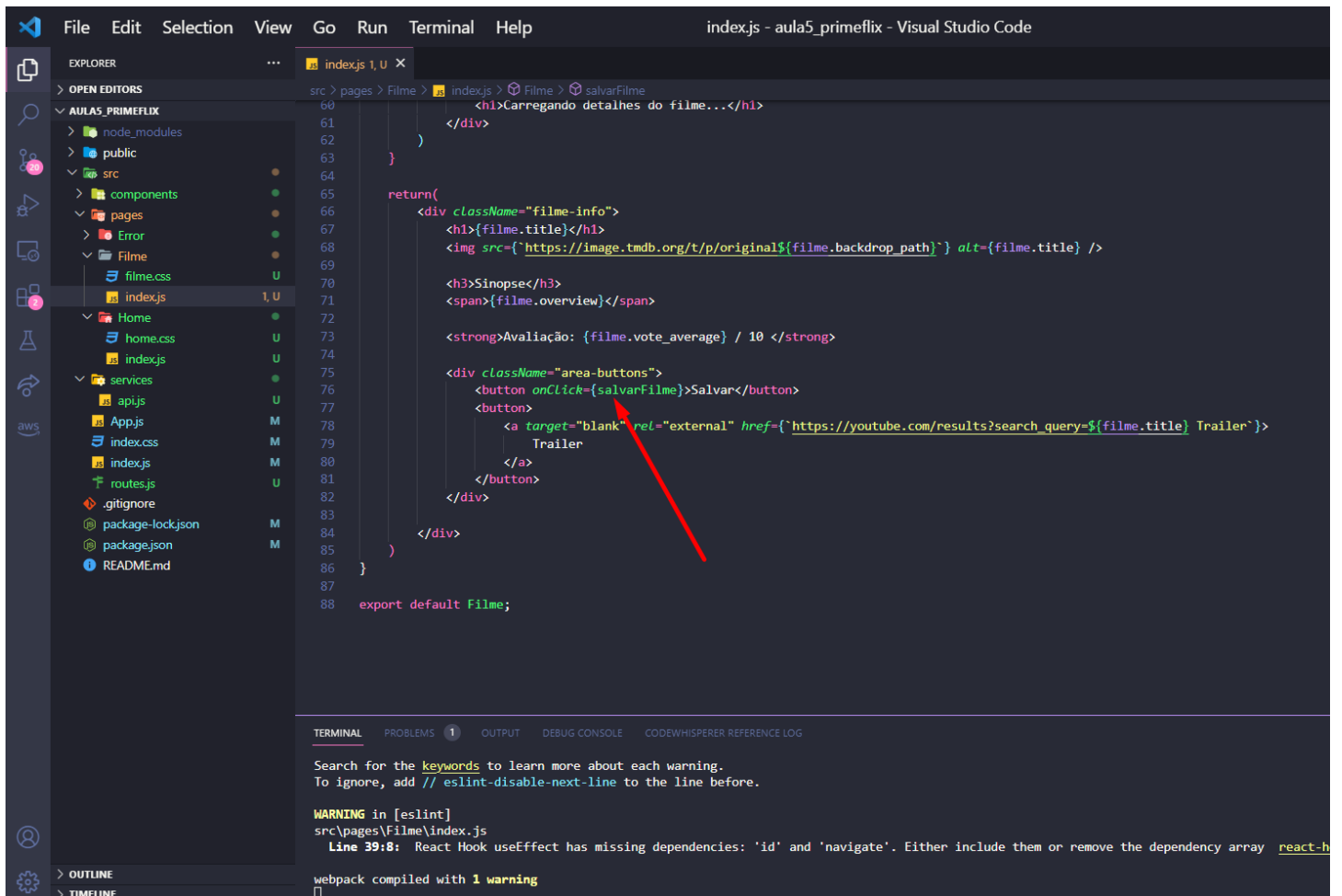
TERMINAL PROBLEMS 1 OUTPUT DEBUG CONSOLE CODEWHISPERER REFERENCE LOG

Search for the **keywords** to learn more about each warning.
To ignore, add `// eslint-disable-next-line` to the line before.

WARNING in [eslint]
src\pages\Filme\index.js
Line 39:8: React Hook useEffect has missing dependencies: 'id' and 'navigate'. Either include them or remove the dependency array [rea](#)

webpack compiled with 1 warning

31 – Após criar a função, basta chamar a mesma função dentro do botão Salvar por meio do onClick:



```
60 <h1>Carregando detalhes do filme...</h1>
61 </div>
62 )
63 }
64
65 return(
66   <div className="filme-info">
67     <h1>{filme.title}</h1>
68     <img src={`https://image.tmdb.org/t/p/original${filme.backdrop_path}`} alt={filme.title} />
69
70     <h3>Sinopse</h3>
71     <span>{filme.overview}</span>
72
73     <strong>Avaliação: {filme.vote_average} / 10 </strong>
74
75     <div className="area-buttons">
76       <button onClick={salvarFilme}>Salvar</button>
77       <button>
78         <a target="blank" rel="external" href={`https://youtube.com/results?search_query=${filme.title} Trailer`}>
79           Trailer
80         </a>
81       </button>
82     </div>
83   </div>
84 )
85 }
86
87 export default Filme;
```

Search for the **keywords** to learn more about each warning.
To ignore, add `// eslint-disable-next-line` to the line before.

WARNING in [eslint]
src\pages\Filme\index.js
Line 39:8: React Hook useEffect has missing dependencies: 'id' and 'navigate'. Either include them or remove the dependency array [react-h](#)

webpack compiled with 1 warning

32 – Vamos testar, no navegador vá em Application → Local Storage → e no endereço localhost atual da nossa aplicação. Ao clicar em Salvar, ele irá criar uma chave chamada @primeflix que implementamos anteriormente, e dentro dessa chave criar um array tendo o filme como um objeto desse array.

The screenshot shows a web browser at `localhost:3000/filme/502356#` displaying a movie page for "Super Mario Bros.: O Filme". The page includes a synopsis, a rating of 7.492 / 10, and a "Salvar" button. The Chrome DevTools Application tab is open, showing the Local Storage. A key named "@primeflix" is highlighted, containing an array of movie objects. A red arrow points from the "Salvar" button to the "@primeflix" key in the storage.

Prime Flix Meus Filmes

Super Mario Bros.: O Filme

Sinopse
Mario é um encanador junto com seu irmão Luigi. Um dia, eles vão parar no reino dos cogumelos, governado pela Princesa Peach, mas ameaçado pelo rei dos Koopas, que faz de tudo para conseguir reinar em todos os lugares.

Avaliação: 7.492 / 10

Salvar Trailer

Application

- Manifest
- Service Workers
- Storage
 - Local Storage
 - `http://localhost:3000`
 - `@primeflix`
 - `[{"adult": false, "backdrop_path": "/9n2tUBpIPbgR2ca05hS5CKXwP2...", "budget": 100000000, "genres": [{"id": 16, "name": "Animação"}, {"id": 12, "name": "Aventura"}, {"id": 35, "name": "Comédia"}, {"id": 28, "name": "Família"}, {"id": 10751, "name": "Fantasia"}, {"id": 14, "name": "Terror"}, {"id": 878, "name": "Sci-Fi"}], "homepage": "https://cuevana3.it/pelicula/the-super-mario-bros-movie-2023", "id": 502356, "imdb_id": "tt6718170", "original_language": "en", "original_title": "The Super Mario Bros. Movie", "overview": "Mario é um encanador junto com seu irmão Luigi. Um dia, eles vão parar no reino dos cogumelos, governado pela Princesa Peach, mas ameaçado pelo rei dos Koopas, que faz de tudo para conseguir reinar em todos os lugares.", "popularity": 13700.561, "poster_path": "/i9XdXHsFrcqLkRWSF1co0Ho4R39.jpg", "production_companies": [{"id": 33, "logo_path": "/81vHyhjr8oUK00y2dKXoALWKC..."}]]`

33 – Perceba que agora toda vez que clicamos em Salvar em um filme um novo objeto no array atual é adicionado.

The screenshot shows a web browser at the URL `localhost:3000/filme/594767`. The page displays a movie entry for "Shazam! Fúria dos Deuses" with a poster image. Below the poster, there is a synopsis and a rating of 6.948 / 10. At the bottom of the movie entry, there are two buttons: "Salvar" and "Trailer".

The Chrome DevTools Application panel is open, showing the state of the application. The left sidebar lists various storage areas: Application, Local Storage, Session Storage, IndexedDB, Web SQL, Cookies, Trust Tokens, Interest Groups, and Shared Storage. The right pane shows the state of the application, with a table of objects. A red arrow points to the second object in the array, which is a JSON object with properties `adult`, `backdrop_path`, and `belongs_to_collection`.

Key	Value
<code>person</code>	<code>Márcio</code>
<code>@tarefa</code>	<code>["Pagar a conta de luz","Estudar Programação","Enviar a tarefa","..."]</code>
<code>@primeflix</code>	<code>[{"adult":false,"backdrop_path":"/9n2tJBp1PbgR2ca05hS5CKXwP2c..."}]</code>

The array contains two objects, both with `adult: false` and `backdrop_path` pointing to a specific image. The second object is highlighted with a red arrow.

[CSS property documentation in the Styles pane](#)

Get information about any CSS property by hovering



34 – E caso o usuário clique novamente, um alert o avisa e nada acontece com nosso array.

localhost:3000 diz
FILME JÁ ADICIONADO

Storage

- Local Storage
 - http://localhost:3000
- Session Storage
 - http://localhost:3000
- IndexedDB
- Web SQL
- Cookies
- Trust Tokens
- Interest Groups
- Shared Storage

@primeflix

```
[{"adult": false, backdrop_path: "/9n2tJ8p1PbgR2ca05hS5CKXwP2c.jpg", belong: 0, ...}, {"adult": false, backdrop_path: "/wybmSmviUXx1BmX44gtpow5Y9TB.jpg", ...}]
```

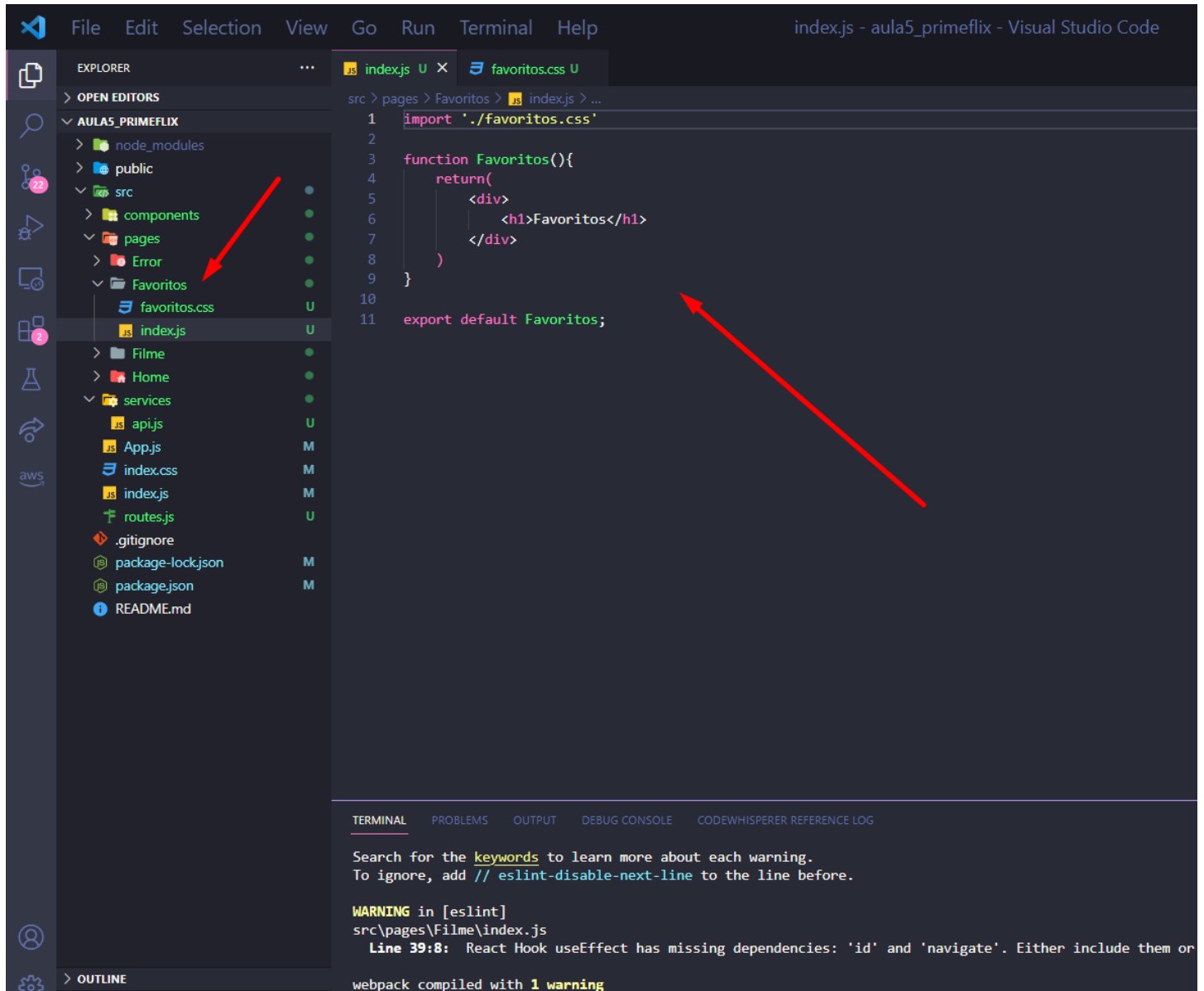
[CSS property documentation in the Styles pane](#)

Get information about any CSS property by hovering

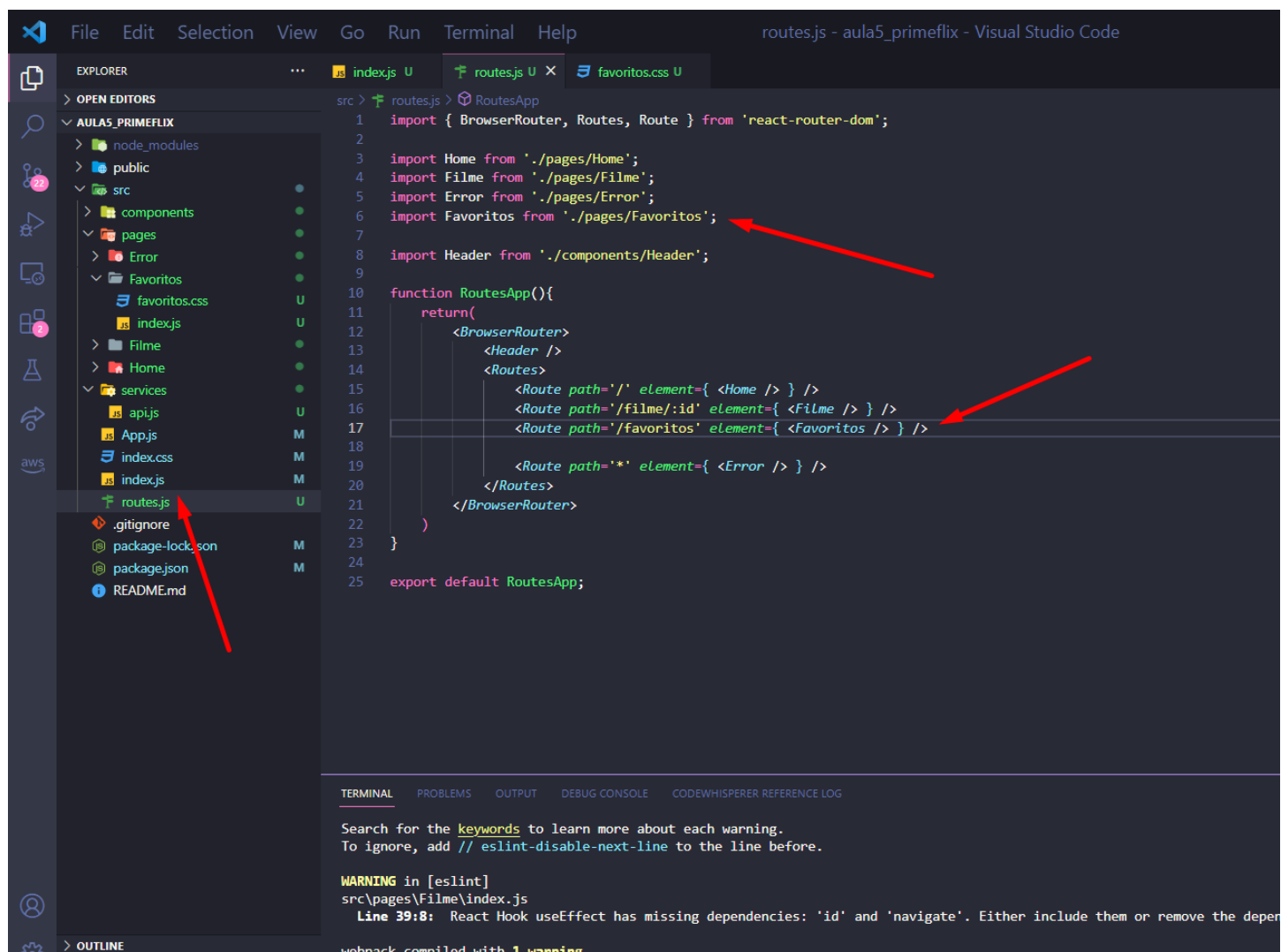


Construindo a página de favoritos

36 – Crie uma nova pasta chamada Favoritos dentro de pages, dentro de Favoritos crie um index.js e coloque a função padrão como abaixo, crie também um arquivo chamado favoritos.css que por hora será vazio.



37 – Vamos agora criar a rota dessa página, dentro de routes.js importe o componente Favoritos (linha 6) e adicione uma nova rota (linha 17).



```
src > routes.js > RoutesApp
1  import { BrowserRouter, Routes, Route } from 'react-router-dom';
2
3  import Home from './pages/Home';
4  import Filme from './pages/Filme';
5  import Error from './pages/Error';
6  import Favoritos from './pages/Favoritos';
7
8  import Header from './components/Header';
9
10 function RoutesApp(){
11   return(
12     <BrowserRouter>
13       <Header />
14       <Routes>
15         <Route path="/" element={ <Home /> } />
16         <Route path="/filme/:id" element={ <Filme /> } />
17         <Route path="/favoritos" element={ <Favoritos /> } />
18
19         <Route path="*" element={ <Error /> } />
20       </Routes>
21     </BrowserRouter>
22   )
23 }
24
25 export default RoutesApp;
```

Search for the **keywords** to learn more about each warning.
To ignore, add `// eslint-disable-next-line` to the line before.

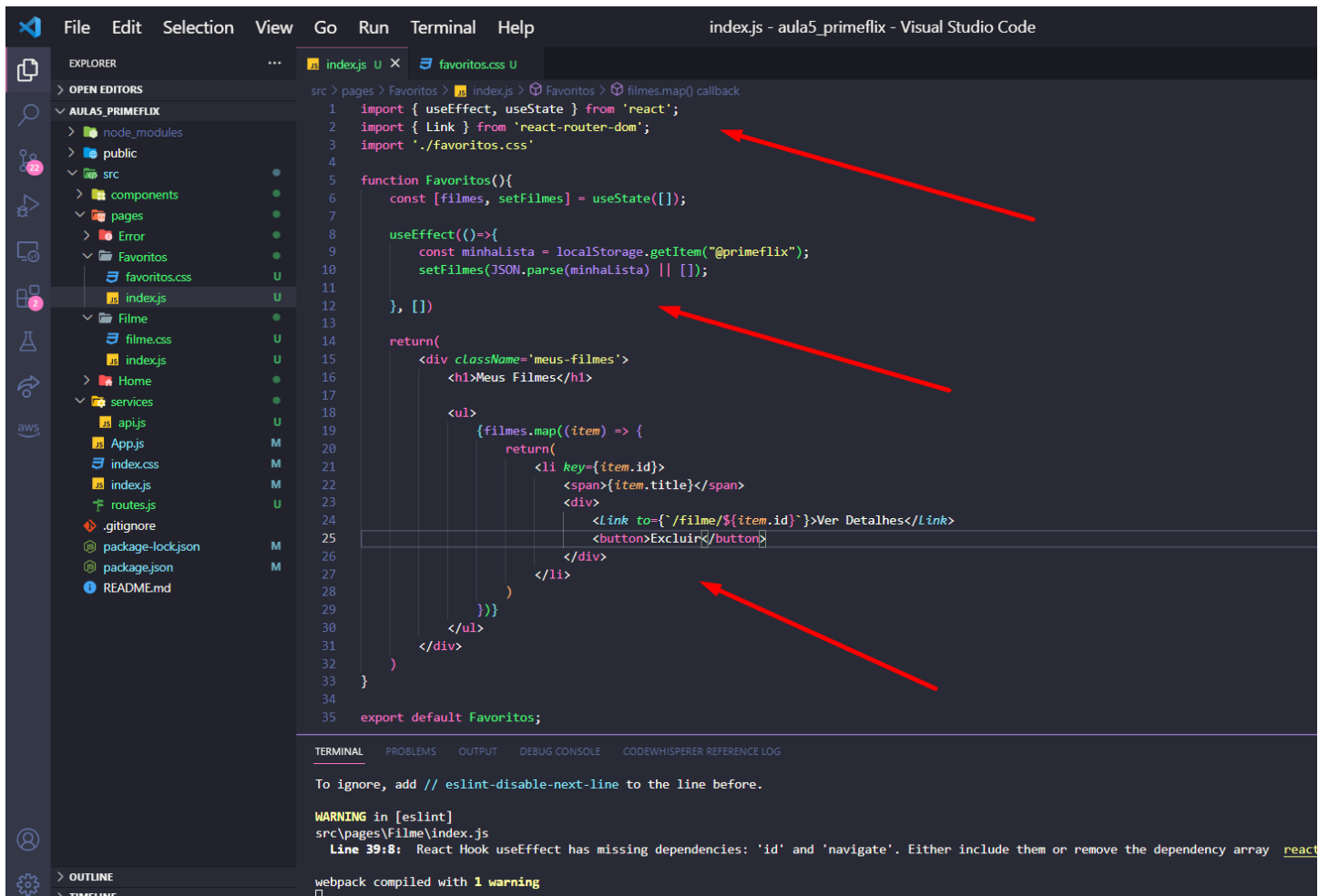
WARNING in [eslint]
src\pages\Filme\index.js
Line 39:8: React Hook useEffect has missing dependencies: 'id' and 'navigate'. Either include them or remove the deper

webpack compiled with 1 warning

38 – Teste, agora ao clicar no botão Meus Filmes ele deve exibir por hora nosso h1 Favoritos.



39 – Implemente agora a página dos Meus Filmes buscando um dado salvo no Local Storage e exibindo na forma de lista na página.



```
src > pages > Favoritos > indexjs > Favoritos > filmes.map() callback
1 import { useEffect, useState } from 'react';
2 import { Link } from 'react-router-dom';
3 import './favoritos.css'
4
5 function Favoritos(){
6   const [filmes, setFilmes] = useState([]);
7
8   useEffect(()=>{
9     const minhalista = localStorage.getItem("@primeflix");
10    setFilmes(JSON.parse(minhalista) || []);
11  }, [])
12
13  return(
14    <div className='meus-filmes'>
15      <h1>Meus Filmes</h1>
16
17      <ul>
18        {filmes.map((item) => {
19          return(
20            <li key={item.id}>
21              <span>{item.title}</span>
22              <div>
23                <Link to={` /filme/${item.id}`}>Ver Detalhes</Link>
24                <button>Excluir</button>
25              </div>
26            </li>
27          )
28        })}
29      </ul>
30    </div>
31  )
32
33 }
34
35 export default Favoritos;
```

TERMINAL

To ignore, add // eslint-disable-next-line to the line before.

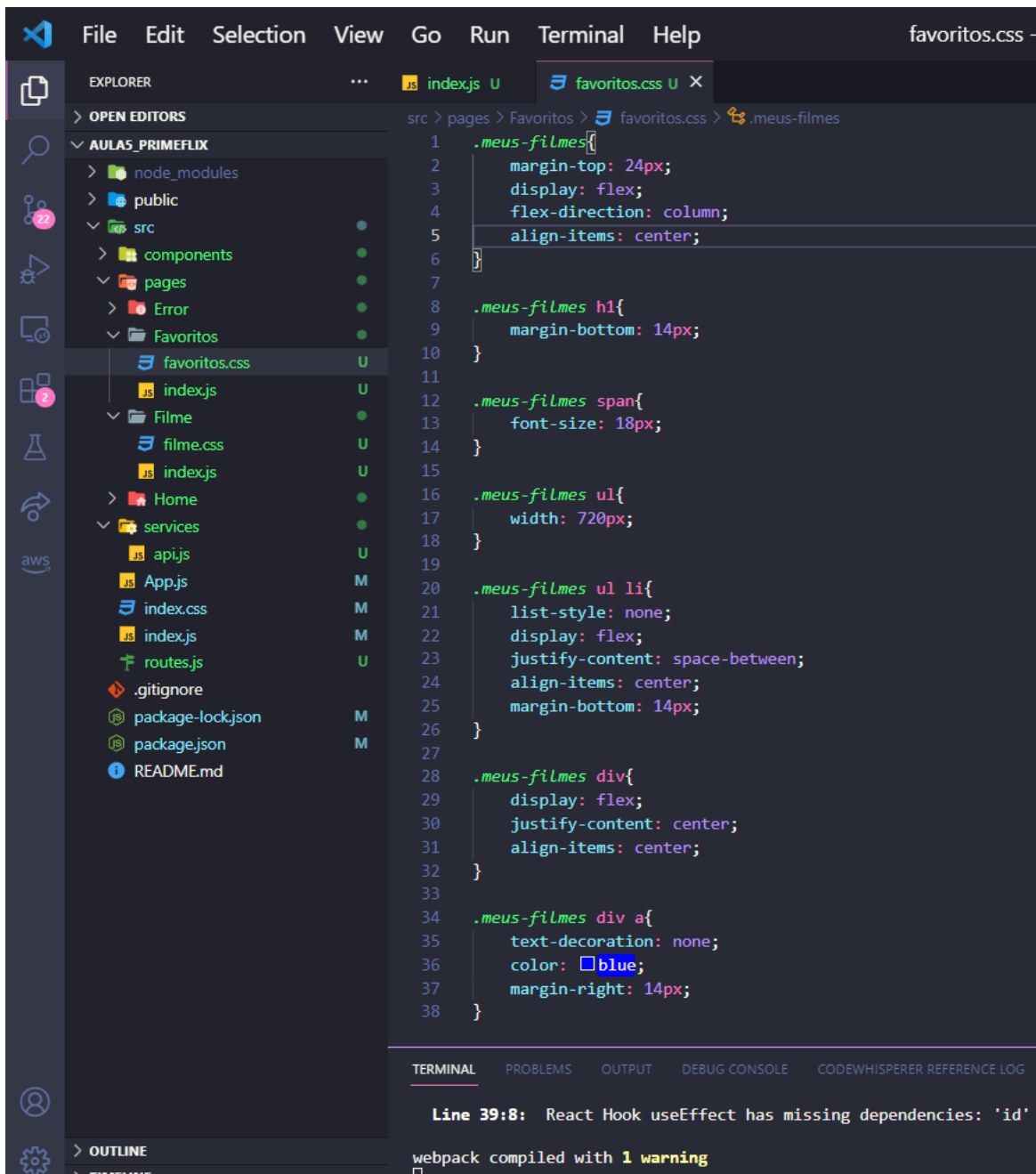
WARNING in [eslint]
src\pages\Filme\index.js
Line 39:8: React Hook useEffect has missing dependencies: 'id' and 'navigate'. Either include them or remove the dependency array [react](#)

webpack compiled with 1 warning

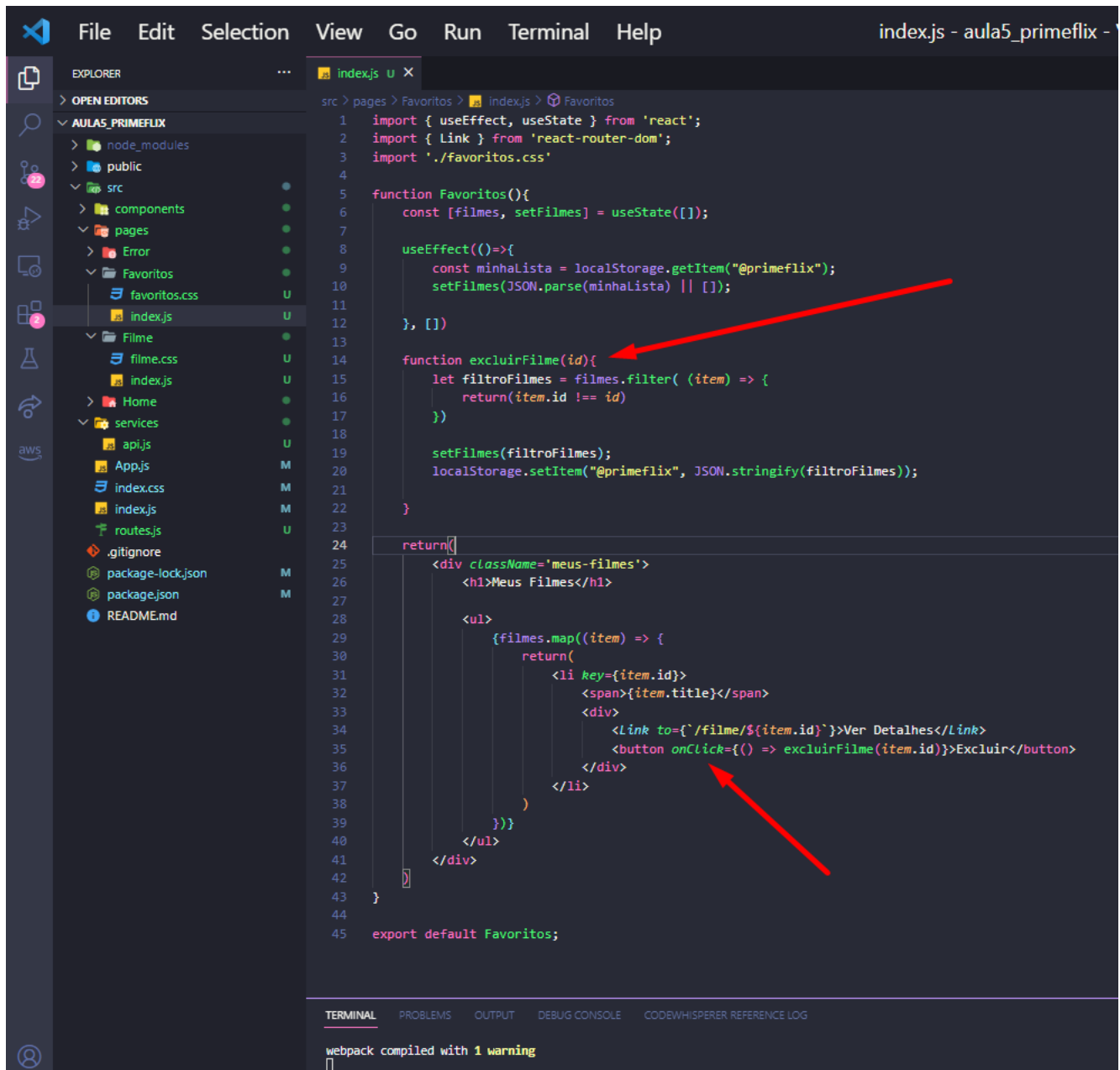
40 – Esse será o resultado atual sem estilização.



41 – Segue a sugestão de estilização da página favoritos.



42 – Vamos agora dar funcionalidade ao nosso botão excluir e assim permitir retirar um filme dos favoritos. Com isso vamos criar uma função chamada `excluirFilmes` que irá manipular o array do Local Storage e retirar o elemento passando como parametro seu id.



```
1 import { useEffect, useState } from 'react';
2 import { Link } from 'react-router-dom';
3 import './favoritos.css'
4
5 function Favoritos(){
6   const [filmes, setFilmes] = useState([]);
7
8   useEffect(()=>{
9     const minhaLista = localStorage.getItem("@primeflix");
10    setFilmes(JSON.parse(minhaLista) || []);
11  }, [])
12
13  function excluirFilme(id){
14    let filtroFilmes = filmes.filter( (item) => {
15      return(item.id !== id)
16    })
17
18    setFilmes(filtroFilmes);
19    localStorage.setItem("@primeflix", JSON.stringify(filtroFilmes));
20  }
21
22  return(
23    <div className='meus-filmes'>
24      <h1>Meus Filmes</h1>
25
26      <ul>
27        {filmes.map((item) => {
28          return(
29            <li key={item.id}>
30              <span>{item.title}</span>
31              <div>
32                <Link to={` /filme/${item.id}`}>Ver Detalhes</Link>
33                <button onClick={() => excluirFilme(item.id)}>Excluir</button>
34              </div>
35            </li>
36          )
37        })}
38      </ul>
39    </div>
40  )
41
42 }
43
44 export default Favoritos;
```

webpack compiled with 1 warning

43 – Para testar, clique no botão excluir de qualquer filme e veja se ele irá sair do array do Local Storage.

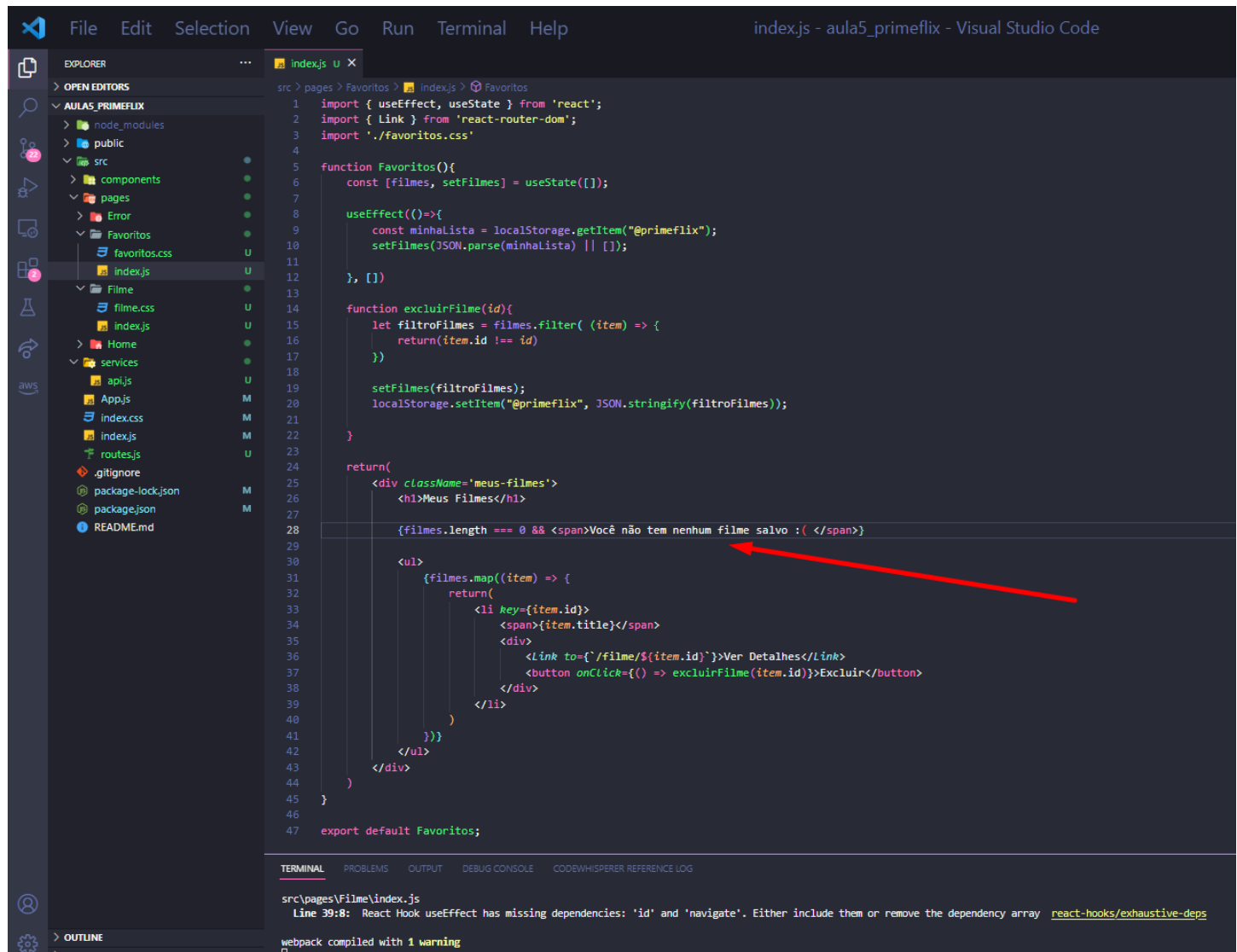
The screenshot shows a web browser at `localhost:3000/favoritos` displaying the 'Prime Flix' application. The page has a header with the 'Prime Flix' logo and a 'Meus Filmes' tab. Below the header, there is a section titled 'Meus Filmes' containing a list of movies: 'Super Mario Bros.: O Filme' and '65: Ameaça Pré-Histórica'. Each movie entry has a 'Ver Detalhes' link and an 'Excluir' button. A red arrow points to the 'Excluir' button for '65: Ameaça Pré-Histórica'.

The Chrome DevTools Application panel is open on the right, showing the 'Storage' tab. The 'Local Storage' section is expanded, showing a single entry for `http://localhost:3000/`. The entry's value is an array of two objects, each representing a movie with properties like `adult` and `backdrop_path`. A red arrow points to the second object in the array, which corresponds to the movie being targeted for deletion.

Key	Value
@primeflix	{ "adult": false, "backdrop_path": "/9n2tJBpIPbgR2ca05hS..." }
peessoa	Márcio
@tarefa	["Pagar a conta de luz", "Estudar Programação..."]

```
[{"adult": false, "backdrop_path": "/9n2tJBpIPbgR2ca05hS...", "id": 1}, {"adult": false, "backdrop_path": "/9n2tJBpIPbgR2ca05hS...", "id": 2}]
```

44 – E se não tiver nenhum filme salvo, podemos melhorar a UX? Faça uma verificação no tamanho do array atual de filmes e caso ele esteja vazio, indicando assim que nada foi salvo no Local Storage, mostre uma mensagem para o usuário.



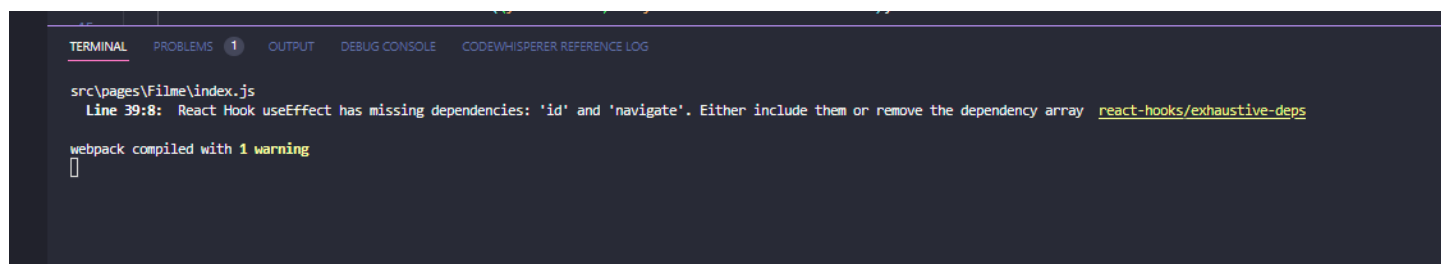
```
1 import { useEffect, useState } from 'react';
2 import { Link } from 'react-router-dom';
3 import './favoritos.css'
4
5 function Favoritos(){
6   const [filmes, setFilmes] = useState([]);
7
8   useEffect(()=>{
9     const minhaLista = localStorage.getItem("@primeflix");
10    setFilmes(JSON.parse(minhaLista) || []);
11
12  }, [])
13
14  function excluirFilme(id){
15    let filtroFilmes = filmes.filter( (item) => {
16      return(item.id !== id)
17    })
18
19    setFilmes(filtroFilmes);
20    localStorage.setItem("@primeflix", JSON.stringify(filtroFilmes));
21  }
22
23  return(
24    <div className='meus-filmes'>
25      <h1>Meus Filmes</h1>
26
27      {filmes.length === 0 && <span>Você não tem nenhum filme salvo :( </span>}
28
29      <ul>
30        {filmes.map((item) => {
31          return(
32            <li key={item.id}>
33              <span>{item.title}</span>
34              <div>
35                <Link to={` /filme/${item.id}`}>Ver Detalhes</Link>
36                <button onClick={() => excluirFilme(item.id)}>Excluir</button>
37              </div>
38            </li>
39          )
40        })}
41      </ul>
42    </div>
43  )
44 }
45
46 export default Favoritos;
```

src/pages/Favoritos/index.js

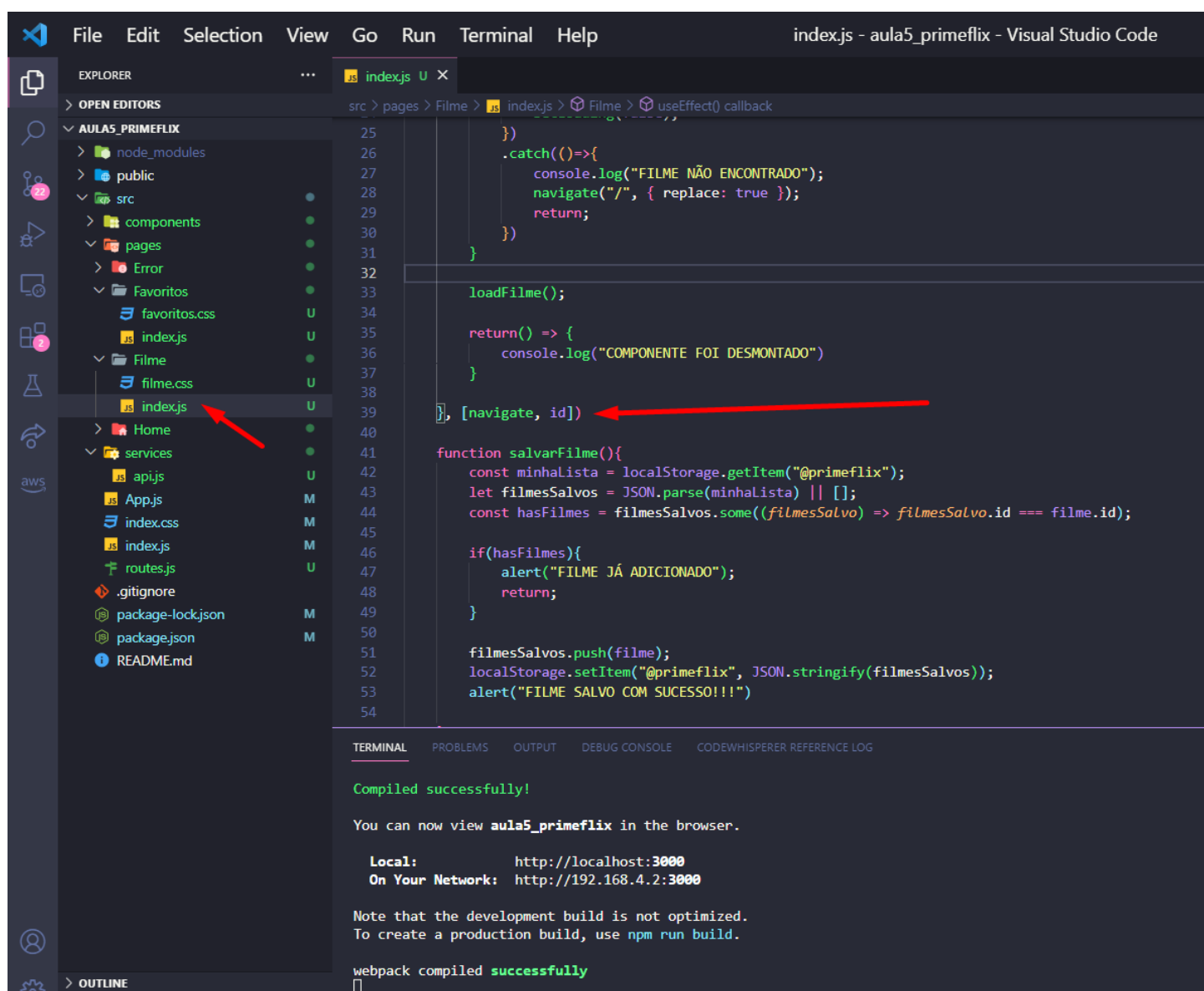
Line 39:8: React Hook useEffect has missing dependencies: 'id' and 'navigate'. Either include them or remove the dependency array [react-hooks/exhaustive-deps](#)

webpack compiled with 1 warning

45 – Caso seu projeto esteja exibindo essa mensagem de alerta, faça a seguinte correção.



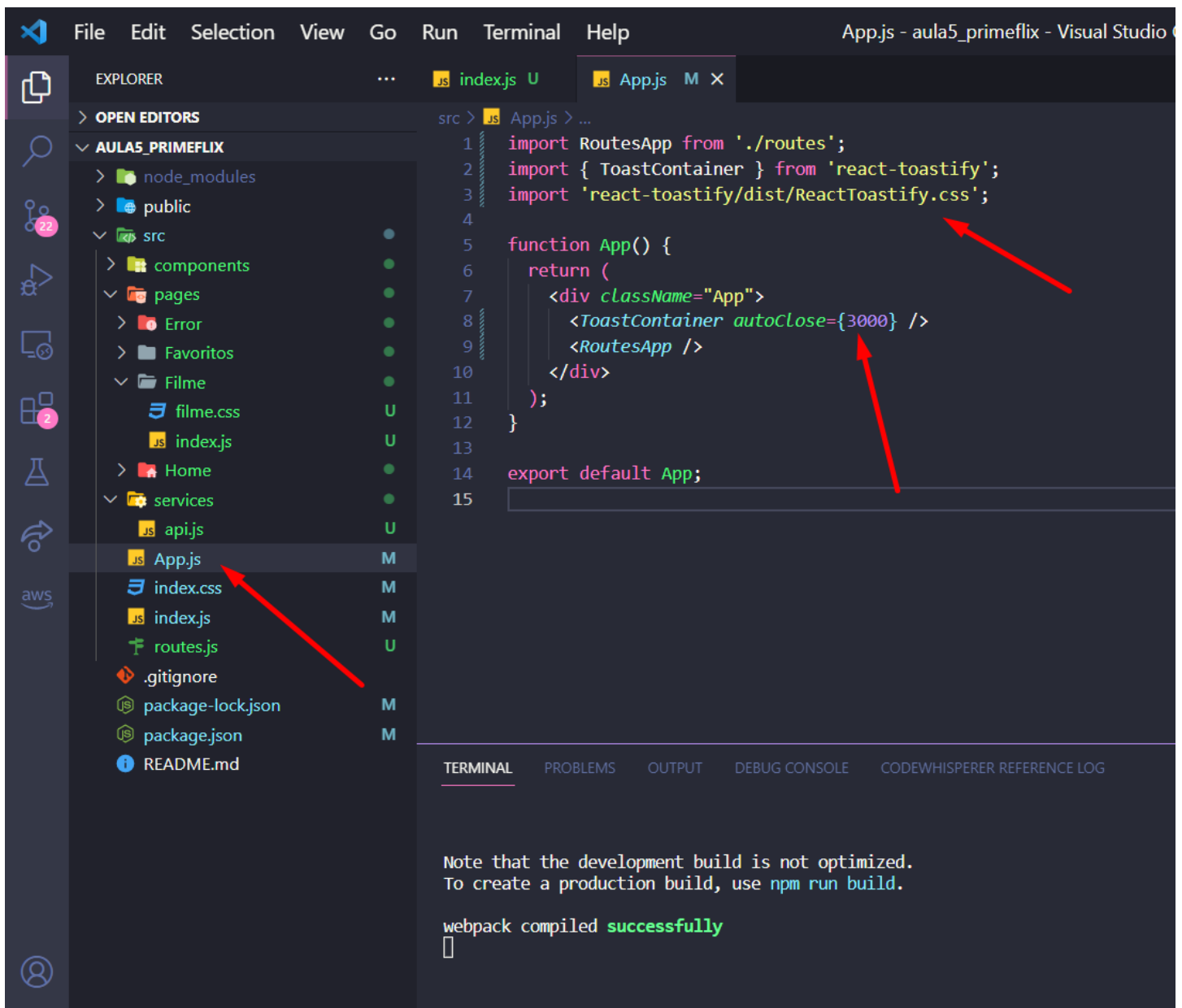
46 – Vá até o `index.js` do Filme e na linha 39, dentro do objeto vazio do `useEffect`, adicione o `navigate` e `id`. Isso irá corrigir o projeto.



47 – Vamos agora instalar uma biblioteca para estilizar nossos alerts. A react-toastify.

```
Note that the development build is not optimized.  
To create a production build, use npm run build.  
  
webpack compiled successfully  
^CDeseja finalizar o arquivo em lotes (S/N)? s  
PS C:\Users\mfunes\Desktop\Fatec\Web III\react\aula5_primeflix> npm i --save react-toastify  
AWS  CodeWhisperer
```

48 – Após a instalação, vá no arquivo App.js e faça a importação desse novo componente.



49 – Vá até o index.js de Filmes, faça a importação do toast (linha 6).

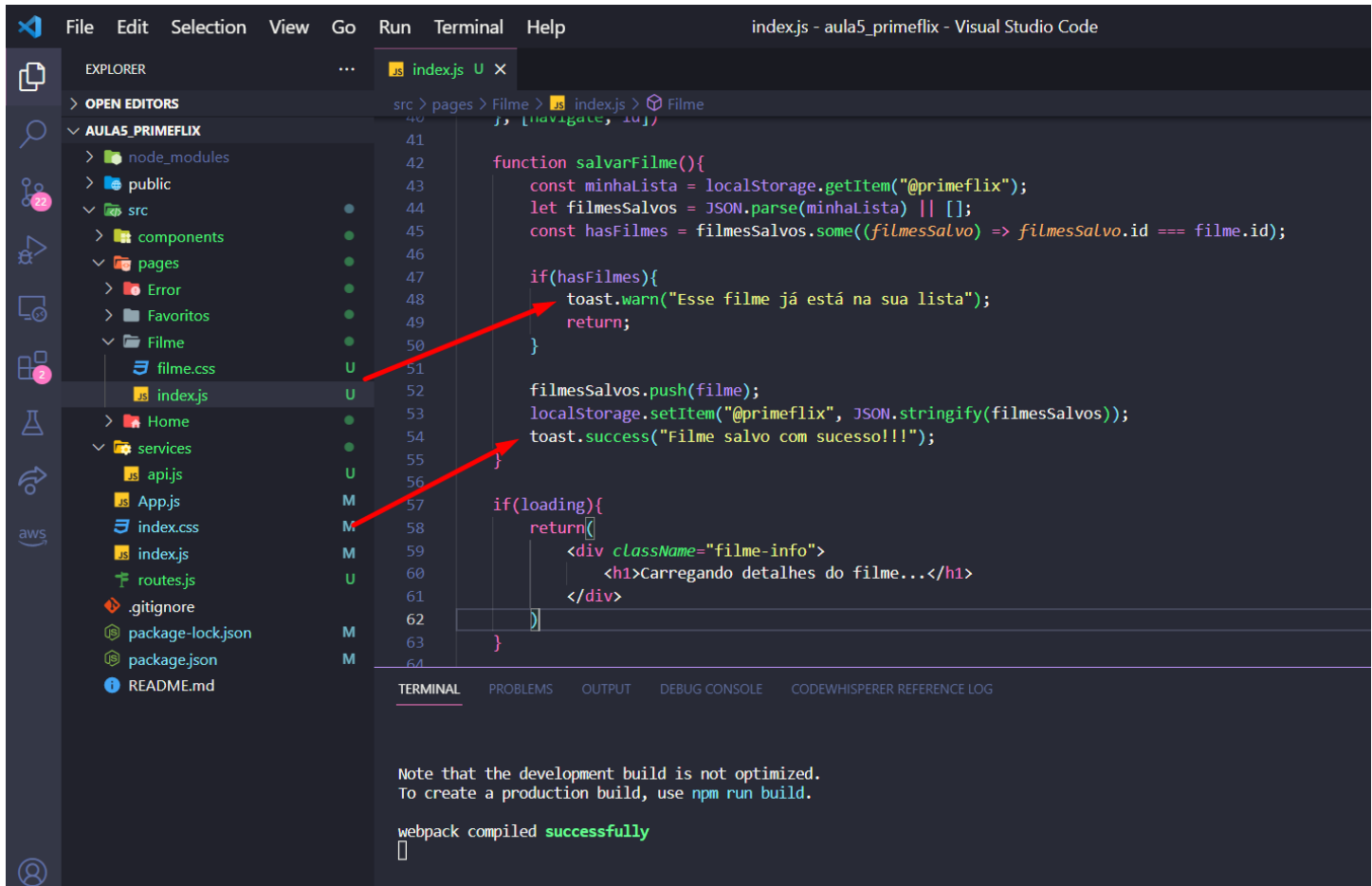
The screenshot displays the Visual Studio Code interface with the following components:

- Explorer:** Shows the project structure. The 'AULAS_PRIMEFLIX' folder is expanded, showing 'src' > 'pages' > 'Filme'. The 'index.js' file in the 'Filme' directory is selected and highlighted with a red arrow.
- Editor:** Displays the content of 'index.js'. The file starts with imports for 'react', 'react-router-dom', and './filme.css'. Line 6, `import { toast } from "react-toastify";`, is highlighted with a red arrow. The rest of the file defines a `Filme` function using `useState` and `useEffect` to fetch movie data from an API.
- Terminal:** Shows the output of an `npm audit` command. It reports that 2 packages were added and 1499 packages were audited in 1m. It also lists 6 high severity vulnerabilities and provides instructions to run `npm audit fix --force` to address all issues.

50 – Agora onde tivermos alert na nossa aplicação, podemos trocar por toast e algum parametro. O success mostra uma mensagem verde já o warn uma laranja. Faça a modificação abaixo.

Veja mais opções de uso na documentação:

<https://fkhadra.github.io/react-toastify/introduction>



```
40
41
42 function salvarFilme(){
43   const minhaLista = localStorage.getItem("@primeflix");
44   let filmesSalvos = JSON.parse(minhaLista) || [];
45   const hasFilmes = filmesSalvos.some((filmesSalvo) => filmesSalvo.id === filme.id);
46
47   if(hasFilmes){
48     toast.warn("Esse filme já está na sua lista");
49     return;
50   }
51
52   filmesSalvos.push(filme);
53   localStorage.setItem("@primeflix", JSON.stringify(filmesSalvos));
54   toast.success("Filme salvo com sucesso!!!");
55 }
56
57 if(loading){
58   return(
59     <div className="filme-info">
60       <h1>Carregando detalhes do filme...</h1>
61     </div>
62   )
63 }
64
```

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE CODEWHISPERER REFERENCE LOG

Note that the development build is not optimized.
To create a production build, use `npm run build`.

webpack compiled **successfully**

51 – Pronto, agora temos uma aplicação bem mais profissional :D
Espero que você tenha gostado!

← → ↻ ⓘ localhost:3000/filme/603692

Prime Flix

Filme salvo com sucesso!!!

John Wick 4: Baba Yaga



Sinopse
Com o preço por sua cabeça cada vez maior, John Wick leva sua luta contra a alta mesa global enquanto procura os jogadores mais poderosos do submundo, de Nova York a Paris, de Osaka a Berlim.

Avaliação: 7.993 / 10

Salvar Trailer